

Syllabus

Module No.	Unit No.	Details	No. of Hrs.	CO
1	Introduction to System software			
	1.1	Concept, introduction to various system programs such as assemblers, loaders, linkers, macro processors, compilers, interpreters, operating systems, device drivers	09	CO 1
	1.2	Operating System Objectives and Functions, The Evolution of Operating Systems		
	1.3	Operating system structures in detail		
	1.4	System Calls		
	1.5	Linux Kernel and Shell		
	1.6	System boot		
#Self-Learning : OS Design Considerations for Multiprocessor and Multicore architectures				
2	Process Concept and scheduling			
	2.1	Process: Concept of a Process, Process States, Process Description, Process Control Block, Operations on Processes. Threads: Definition and Types, Concept of Multithreading	10	CO 2
	2.2	Multicore processors and threads. Scheduling: Uniprocessor Scheduling - Types of Scheduling: Preemptive and, Non-preemptive, Scheduling Algorithms: FCFS, SJF, SRTN, Priority based, Round Robin, Multilevel Queue scheduling.		
	2.3	Thread: Introduction to thread, Multithreading models		
# Self-Learning : WINDOWS Booting Process, Linux Scheduling				
3	Process Concurrency			
	3.1	Concurrency: Principles of Concurrency, InterProcess Communication, Process/Thread Synchronization.	12	CO 3
	3.2	Mutual Exclusion: Software Approaches – Peterson Solution, Software Support, Hardware Support, Operating System Support (Semaphores and Mutex), Programming Language Support (Monitors), Classical synchronization problems: Readers/Writers Problem, Producer and Consumer problem.		
	3.3	Principles of Deadlock: Conditions and Resource Allocation Graphs, Deadlock Prevention, Deadlock Avoidance: Banker's		
	3.4	Algorithm for Single & Multiple Resources, Deadlock Detection and Recovery. Dining Philosophers Problem.		
4	Input output and file management			
	4.1	File Management: Overview, File Organization and Access, File Directories, File Sharing, Secondary Storage Management, Linux Virtual File System, Inode Structure	09	CO 4
	4.2	I/O Management and Disk Scheduling: I/O Devices, Organization of the I/O Function, Operating System Design Issues, I/O Buffering, Disk Scheduling algorithm: FCFS, SSTF, SCAN, CSCAN, LOOK, CLOOK. Disk Management, Linux		

Module No.	Unit No.	Details	No. of Hrs.	CO
		I/O.		
5	Storage management			
	5.1	Main Memory: Background, Swapping, Contiguous Memory Allocation, Buffering	05	CO 5
#Self-Learning Component: Android OS, Cloud OS, Contemporary issues and solutions in Memory Management				
Total			45	

Module 1

Introduction to System Software

Syllabus

			HRS.	
1	Introduction to System software			
	1.1 Concept, introduction to various system programs such as assemblers, loaders, linkers, macro processors, compilers, interpreters, operating systems, device drivers Operating System Objectives and Functions,		09	CO 1
	1.2 The Evolution of Operating Systems			
	1.3 Operating system structures in detail			
	1.4 System Calls			
	1.5 Linux Kernel and Shell			
	1.6 System boot			

Operating System

Definition: OS is a system software that controls execution of programs and acts as an interface between applications and computer hardware.

Objectives:

1. Convenience: makes the machine convenient to use
2. Helps in the efficient use of system resources
3. Permits effective development, testing, and introduction of new system functionalities without interfering with service

Services provided by an OS

- **Program Development**
The OS provides tools like editors and debuggers. These help programmers write, edit, and test programs easily.
- **Program Execution**
The OS loads the program and required data into main memory. It initializes input/output devices and files, then starts the program execution.
- **Access to I/O Devices**
The OS controls input and output devices like keyboard, mouse, and printer. It hides the hardware details from the user and manages communication with devices.

- **Controlled Access to Files**

The OS manages files by keeping track of file size and free disk space. It also provides protection, especially in multi-user systems, so users cannot access unauthorized files.

- **System Access**

The OS ensures only authorized users can access the system. This is important in public or multi-user environments for security.

- **Error Detection and Response**

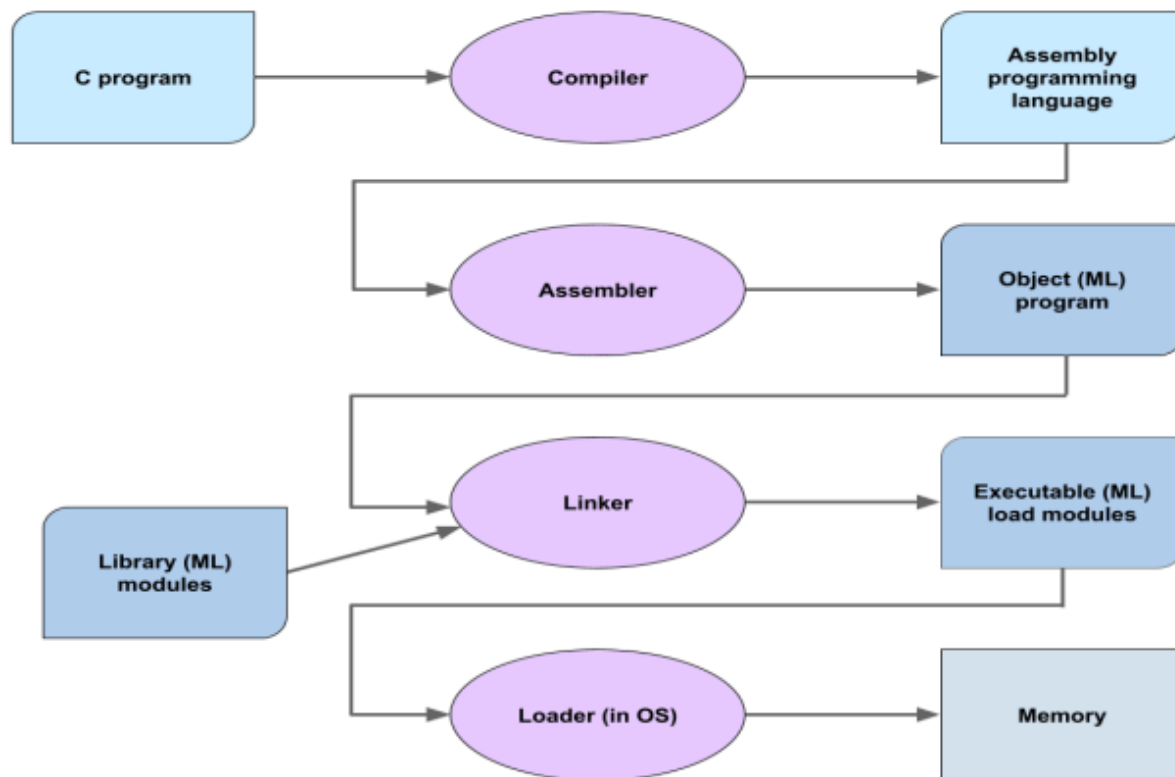
The OS detects errors such as memory problems or device failures. It takes necessary action to prevent system crash or data loss.

- **Accounting**

The OS keeps records of resource usage like CPU time and memory usage. This helps in billing in multi-user systems and improves future system performance.

Mnemonic: Development, execution, IO (DIE). File and system access, error and accounting (SAFE)

High level language to Machine Language



Dr. Prasanna Shete KJSCE 2025-26

C Program (HLL)

Program is written in C language. It is human-readable.

Compiler

Compiler converts C code into **Assembly language**. It checks errors and translates instructions.

Assembler

Assembler converts assembly code into **Object code (Machine Language)**. This is in binary form but not yet complete.

Linker

Linker combines object code with **library files**. It creates a complete **executable file**.

Loader (in OS)

Loader loads the executable into **main memory (RAM)**. Program is now ready to run.

C Program → Compiler → Assembler → Linker → Loader → Memory → Execution

System Programming

System Programming: The building of Systems Software (like Windows, macOS, boot loaders, antivirus etc) using System Programming Languages is called system programming. It serves as an interface between the users and the hardware.

There are two types of system programming

1. System software
2. Application software

System Software

- Set of programs that control and manage the operations of the computer hardware.
- Helps application programs execute correctly.
- Examples include macro processors, compilers, interpreters, assemblers, linkers (program that combines Object code with library files), loaders(program that loads the code into the main memory), OS, and device drivers (controls a specific hardware device.)

1. Macroprocessors

Pre-processors are programs that process the source code before compilation.

There are four types of directives here:

a. Macros

A sequence of computing instructions available as a single program statement.

Reduces chances of error.

The #define directive is used to define a macro

Macros are not terminated by a semicolon.

For understanding:

```
#define MOD 100000007
#define yes cout << "YES" << endl;
int main(){
    int x = some number;
    cout << x%MOD << endl;
    yes //prints yes
}
```

b. File inclusion

Header files are those that contain the definitions of the predefined functions like printf, scanf, etc.

In order to work with these functions, importing the header files is important.

Syntax:

```
#include <filename.h>
```

When a program becomes large, it is important to divide it into smaller modules for better understanding. Then these user-defined files can be imported like standard header files into the program like usual

```
#include "filename.h"
```

Standard	User defined
File is loaded from the default directory	File is loaded from the current working directory
Only pre-defined header files	User-defined header files

c. Conditional compilation directives

These help compile a specific portion of the program or skip the compilation of a specific portion of the program.

Commands: ifdef and endif

```
#include <stdio.h>
#define YEARS_OLD 10
int main()
{
    #ifdef YEARS_OLD
        printf("TechOnTheNet is over %d years old.\n", YEARS_OLD);
    #endif
    printf("TechOnTheNet is a great resource.\n");
    return 0;
}
```

This ensures that if the YEARS_OLD macro is defined, only then the following statement is printed. The last statement is printed nevertheless since it comes out of the conditional compilation directive.

d. Other (Unimportant)

Undef and pragma are two more directives

Undef is used to undefine a macro we earlier defined

Pragma is used to turn on/off some feature

Compiler

Definition: a software that translates code from one language to another. It is a language processor that reads the source code written in a high level language(HLL) ALL AT ONCE and translates it to machine language(ML).

- It makes the target code efficient and optimized.
- It is only translated into ML if it's free from errors.
- If there are errors in the program, the compiler specifies them with line numbers.

Assembler

Definition: It converts assembly language programs into object files

An object file contains machine instructions, data, and information regarding these instructions.

Interpreter

Definition: It is a software that translates HLL to ML line by line.

- When an error is encountered, execution stops and the interpreter reports the error
- It only moves to the next line if and when the error is resolved.

Compiler	Interpreter
Works on the whole source code at once; creates tokens, checks semantics, generates intermediate code	Reads a statement from the input, executes it, then takes the next statement in sequence.
Reads the whole program despite errors in the earlier part.	Stops reading the instant it meets an error.
Takes a large amount of time to analyse the source code but overall execution time is small.	Takes less time to analyse source code but overall execution time is large.
Debugging is hard as the error can be present anywhere	Debugging is easy since the error is reported as the interpreter meets it.
Generates an intermediate object code	No intermediary code is generated.
Ex: Java, C++, C	Example: Python, Perl

Linker

Definition: A linker is a tool used to link all the parts of the program together for execution (executable machine code).

- It merges all the object files into a single executable file.
- Also called link editor
- It searches and locates the referenced modules in a program/ library routines mentioned in the program.

Loader

Definition: Loads the executable code from disk to RAM

- Allocates space to the executable in the main memory
- Transfers control to the start of the instruction of the program (main() in cpp and c)
- Must reside in the main memory at all times

Operating system

We know that an OS is a system software that controls execution of programs and acts as an interface between applications and computer hardware. It is responsible also for managing resources. When the OS gives CPU to a program, it temporarily loses control. The program runs in user mode. The OS regains control only when the processor generates an interrupt.

OS as a resource manager

1. Manages hardware resources
2. Manages software resources
3. Ensures fair distribution of resources

OS as a User interface/Computer interface

1. Acts as a bridge between users and computer hardware
2. Provides CLI/ GUI/API

Functions of an OS

Spimduf

1. Memory management
2. Process management
3. Device management
4. File management
5. I/O management
6. User interface

7. Security

Kernel

Definition: Kernel is the part of the OS that resides in the main memory. It contains the most frequently used OS functions.

System call

A system call is the function by which a program makes a request to the OS. These are a set of standard functions through which we can establish a direct connection with the OS.

CPU modes

Kernel mode: This mode is for the OS. All instructions are allowed.

User mode: This mode is for the user programs. I/O and some privileged instructions are forbidden.

Program

A set of instructions to perform some task. Example: source code

Process

A program in execution is called a process

Evolution of Operating System

Reasons for Evolution:

Operating systems evolved due to hardware upgrades, introduction of new hardware devices, need for new services, and fixing errors in existing systems.

Stages:

1. Serial Processing

In early systems, only one job was executed at a time.

There was no operating system.

Users interacted directly with hardware.

Errors were indicated by lights and the system ran from a console with toggles and input devices.

Limitations: Scheduling, high setup time

Scheduling refers to the reservation of machine time for operations. It was wasteful in case of idle time and if forceful termination of programs happened.

Setup time was high as loading the compiler, loading HLL, saving the code, linking loading etc took much time. In case of errors, users had to restart from the setup sequence.

2. Simple Batch Systems

Jobs were collected and executed in batches.

A resident **monitor** automatically loaded and executed jobs. The program came back to monitor after execution and monitor automatically loaded the next job. A monitor is a synchronization mechanism in Operating Systems that allows only one process/thread to access shared data at a time.

Reduced idle time of CPU.

Features:

- Timer: prevents a job from monopolizing the system(One job cannot take all the CPU time)
- Privileged instructions: certain ML codes could only be run by the monitor
- Interrupts: Allowed OS to regain control when needed
- File management was made simple
- Memory protection: added a layer of security. One file cannot affect another anymore.

3. Multi-programmed Batch Systems

Uniprogramming: I/O devices are slower than processors and hence they make the processor wait before proceeding with operations.

Multiprogramming: In multiprogramming, if one process is waiting on the I/O devices, the processor can take up the next job which is ready.

Multiple jobs were kept in memory at the same time. CPU switched between jobs to improve utilization. This increased system efficiency.

4. Time-Sharing Systems

Multiple users could access the system simultaneously through terminals.

OS interleaves the execution of each program in short bursts. CPU time was divided into small time slices. This lets the user interact with the computer directly. (Interactive computing)

	Batch Multiprogramming	Time Sharing
Principal objective	Maximize processor use	Minimize response time
Source of directives to operating system	Job control language commands provided with the job	Commands entered at the terminal

OS design structures

Monolithic systems

A monolithic system has no clear structure. The entire OS is one large program.

It is written as a collection of procedures. Any procedure can call any other. Each has a defined interface. There is no information hiding, all procedures are visible to each other. During construction, all procedures are compiled. They are linked into one object file. The system linker binds them together.

Working:

1. The user program makes a kernel call.
2. OS checks which system call is needed.
3. OS calls the required service procedure.
4. Control returns to the user program.

Basic Structure

1. The main program handles system calls.
2. Service procedures execute system calls.
3. Utility procedures support service procedures.

Characteristics

- This has high performance due to direct function calls from the kernel.
- Low modularity since all programs are in a single codebase.
- Maintenance is difficult since change in one part affects all others.
- Low security since one bug can compromise the entire codebase and due to lack of information hiding.

Layered Systems

Layer No.	Name
5	The Operator
4	User programs
3	I/O management
2	Operator-process communication
1	Memory and Drum management
0	Process allocation and multiprogramming

Mnemonic: PAM, memory, communication, io, user prg, operator
Please Make Computers In Universities Operate.

Lowest layer has the hardware

Highest layer has the user interface.

The higher layers use the lower layers in their procedures.

Layer 0: Process Allocation and Multiprogramming

- Deals with allocation of processor: switches between processes based on interrupts/timer expiry
- Provides for the basic multitasking of the CPU

Layer 1: Memory and Drum Management

- responsible for managing memory
- Allocates space for processes in main memory and on the 512K word drum. [the drum is used for holding parts of processes which did not have room to be stored in RAM/main memory]

Layer 2: Operator Process Communication

- Handles communication between a process and an operator console(the device with which the user/OS controls a process)
- It reads user commands. It passes them to the OS.

Layer 3: I/O management

- Manages input/output devices. It controls device communication.
- It also performs buffering. Buffering stores data temporarily. This ensures smooth data transfer between CPU and devices.

Layer 4: User Programs

- This is the layer where the programs written by users live. They don't have to concern themselves with i/o management, memory, process, etc.

Layer 5: Operator

- This layer represents the system operator. It is the highest layer. It interacts directly with the system. It is the location of the operator process (console process). The operator gives commands. System status is monitored here.
- Example: In Windows → Command Prompt / PowerShell.

TRAP Instruction:

A trap is a software interrupt. It transfers control from user mode to a more privileged kernel mode.

OS is organized into concentric rings. Inner layers are more privileged than the outer rings.

Characteristics of Layered OS Architecture

1. Moderate performance: each layer adds overhead
2. Higher modularity than a monolithic system as functionality is divided between layers
3. Maintenance: Easier maintained than monolithic as changes are localised within layers
4. Security: more secure than monolithic but a bug in one layer can lead to the effect travelling to upper layers.

Example: Linux/windows XP

Virtual Machines

Definition: These are time shared systems (OS interleaves processes in bursts). Originally called a conversational monitor system or CMS. Provides multiprogramming and an extended machine with a more convenient interface. The VMM or virtual machine monitor is the heart of the system. It runs on bare hardware and executes multiprogramming. Therefore providing upper layers with multiple virtual machines.

VMs are not extended machines with files and features. Instead, they are copies of the bare hardware including the kernel, user made, io interrupts, and anything in another real machine.

Different VMs can run different operating systems.

Basic: Virtual Machine (VM): A VM is a software-based copy of a real computer.

VM/370: VM/370 was an operating system developed by IBM. It created and managed multiple virtual machines.

Working

1. When a CMS program executes a system call, the call is trapped to the OS inside its own Virtual Machine, not to VM/370.
2. It behaves exactly as if it were running on a real machine instead of a virtual one.
3. After that, CMS issues the normal hardware I/O instruction to read its virtual disk or any required resource.
4. These I/O instructions are then trapped by VM/370. VM/370 performs them as part of its simulation of real hardware.

Client server systems

Principle: this moves most of the functionalities to the higher layers. Tries to make the kernel as minimal as possible. Implements most of the OS functions in the user processes layer.

- To request a service, the user process sends a request to the server process.
- The server operates and sends the result back to the client (user).
- Kernel is responsible for the communication between client and server
- OS is split into parts: each handles one facet of the system.
- Makes management easier
- All servers run in user mode, not in kernel mode. Therefore, they don't have direct access to the hardware.
- If a bug is discovered in the server, the file may crash but the system remains intact.

Modern OS

Microkernel architecture

- In this architecture, only few essential functions are sent to the kernel. These functions include address space, interrupts, interprocess comms, and basic scheduling
- Other functions run in user space, outside the kernel
- **Decoupling between kernel and server:** It simplifies implementation, adds flexibility,

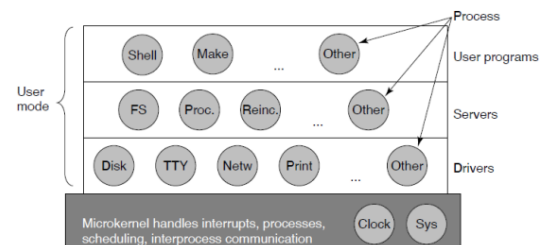


Figure 1-26. Simplified structure of the MINIX system.

and is well suited for a distributed environment (system where multiple computers work together over a network).

- This architecture separates the basic kernel functions from higher level services. The functions specific to the kernel like hardware-software communication, scheduling, etc. happens in the kernel alone and everything else happens in the user space.
- Examples: MINIX and QNX run the most important services in the kernel space, and other services in the user space.
- **Characteristics**
 1. Low performance due to more context switches/more interprocess comms
 2. High modularity since only essential services are kept in the kernel
 3. Easiest to maintain as most services in the user space are updated independently
 4. Highest security since any fault in user space does not affect kernel

Multithreading

- The process executing an application is divided into threads. Can run simultaneously.
- A thread is a dispatchable unit of work. It includes processor context and its own data for a stack where it runs the function calls.
- It executes in sequence and is interruptable: a process can run another thread by interrupting one thread.
- Process is a collection of one or more threads and system resources.
- Advantages
 1. Great control over modularity
 2. Great control over timing of application
 3. Multithreading is useful for apps where tasks are not serialized.

Distributed OS

One OS controlling many computers. This is used in cluster computers/multi computers. Creates an illusion of a single main memory and a secondary memory. The system appears as one single computer to users.

- Supports distributed file systems.
- Enables resource sharing between machines.
- Provides load balancing (work shared across nodes).
- Supports process communication across machines.
- Improves reliability and fault tolerance.

Object oriented OS

- Allows adding modular extensions to a small kernel
- Enables customization without disrupting system integrity

Symmetric Multiprocessing

Computers with multiple processors employ SMP. Instead of one processor doing 10 tasks one by one, you've got 8 processors doing them simultaneously. Way faster.

- Processors share same memory and Io
- Processors are interconnected by communication bus
- All processors can perform the same functions
- The existence of multiple processors is not visible to users: The OS schedules threads and processes on individual processors and initiates synchronization among them automatically.
- The OS of a symmetric multiprocessor schedules threads across all processors and provides functions to exploit the parallelism in the system (make the most use of the parallelism in the system)

OS design considerations for symmetric multiprocessor

1. **Simultaneous threads:** Multiple processors may run processes or threads at the same time. Kernel routines must be re-entrant. This means the same kernel code can run simultaneously on multiple processors.
2. **Scheduling:** Any processor can perform scheduling. So you need to make sure two processors don't accidentally schedule the same thread at the same time. Coordination needed.
3. **Synchronization:** Multiple active processes have potential access to shared address spaces or shared I/O resources. Therefore, enforcing an effective synchronization facility is important.
4. **Memory management:** Each processor has its own cache, so if one updates a value the others might still have the old version sitting in their cache, and the OS has to keep everything in sync. It needs to be smart about actually spreading work across all processors so none of them are sitting idle.
5. **Reliability and fault tolerance:** Should provide graceful degradation in the case of processor failure. The scheduler must recognize that a processor has been lost and restructure the management tables accordingly.

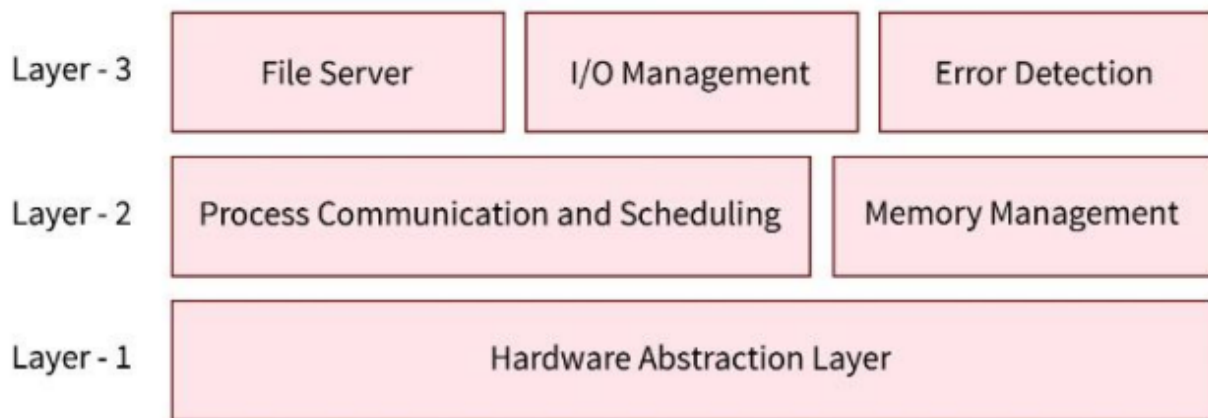
Parameter	Microkernel	Monolithic
Address space	User services and kernel	Both are kept in the

	services have different address space	same address space.
Design and implementation	Complicated OS design	Easy OS design
Size	Smaller microkernel	Larger than microkernel
Functionality	Easier to add and update new functionalities	Difficult to add new functionalities
Coding	More code is required to write this	Less code is needed
Failure	Failure of one component does not affect the working of the whole kernel	Failure anywhere leads to the failure of the entire system.
Processing speed	Low execution speed	High execution speed
Extend (same as functionality point)	Easy to extend	Difficult to extend
Communication	IPC messaging queues are used	Signals and sockets are used
Debugging	Simpler due to modularity	Difficult to debug
Maintenance	Easier to maintain	More time and resources needed: harder to maintain
Message passing** and context switching*	Message passing and context switching is required	Message passing and context switching is not required
Services	The kernel only offers IPC and low level device management	Contains all of the OS's services
example	Mac OSx	Mac OS, Unix

*Context Switching: The CPU switches from one process/thread to another.

**Message Passing: A method for communication between processes.

Hybrid Architecture



Processes that need to be fast and can't afford the overhead of message passing (like scheduling and memory) stays deep in the kernel. The processes that can afford to be more modular and isolated go higher up.

A hybrid architecture combines the elements of both monolithic and microkernel architectures to leverage the benefits of both architectures.

Example: Windows NT

Hybrid architecture:

1. Performance: several components provide specialized services: leads to overall better system performance
2. Modularity: Each layer is responsible for a specific function, developers can work on individual layers independently.
3. Maintenance: Easier maintenance as modules are separated.
4. Security: separating the kernel space from the user space enhances security.

Evolution of OS

Batch processing systems -> Resident monitor -> Single user single tasking -> multiprogramming -> time sharing -> single user multitasking / multiprocessing -> multiuser multiprocessing -> personal computer OS -> network OS -> distributed OS -> Modern OS -> Mobile OS -> Cloud and virtualization

Categories of OS

1. Single user single task: Designed for one user to perform one task at a time. Example MS DOS
2. Single User Multi-tasking: Allows one user to run multiple applications simultaneously. Example microsoft windows

3. Multi-user: Enables multiple users to access a single system's resources simultaneously. Example: Unix/Linux
4. Distributed: Computing resources are distributed across multiple machines but appears as a single system. Example: Google android
5. Multiprocessing: Multiple CPU for higher speed. Example: Linux SMP (Symmetric multiprocessing)
6. Parallel OS: Systems that perform tasks in parallel on multiple processors. Example: Cray OS
7. RTOS
Real time operating system. Processes data and gives output in a defined timeframe. Used in time-critical systems.
Example: Tesla

Multicore

Multicore is a processor that contains multiple cores inside a single CPU chip. Each core works like an independent processor and can execute instructions separately. All the cores share the same main memory and system resources. This allows the system to run multiple processes or threads at the same time, improving performance and efficiency. Multicore processors increase parallel processing, reduce power consumption compared to multiple separate CPUs, and improve the overall speed of the computer system.

Shell

A shell is an interface that allows users to interact with the OS.

Two categories:

1. Command line shell
2. Graphic shell

Command line shell

- Can be accessed by user using the command line interface
- The terminal/cmd prompt is used to type human readable code for execution
- Result is shown on terminal
- Users can store commands in a file and execute them together. Makes for the easy automation of repetitive tasks.
- Harder to use by beginners since shell scripting language and syntax need to be memorized.

Graphic shell

- Helps manipulate programs based on the graphic user interface

- Allows operations like opening, closing, moving, resizing windows.
- Example: Windows/Ubuntu
- No need to type commands for every action
- Easy to user/beginner friendly
- Most GUI systems also have a CLI interface.

The \$ prompt is issued by the shell. A command may be typed after this prompt. Pressing enter makes Shell read your input.

- A \$ prompt : bash(bourne again)/korn/bourne(faster and preferred) shell
- A % prompt : C shell

Using chsh utility: chsh is the utility to change a user's login shell.

Bash command syntax is a superset of the Bourne shell command syntax.

Each shell file has an .sh extension

Shell definition line: tells the system what shell should be used to interpret the script's commands and where the shell is located.

Example:

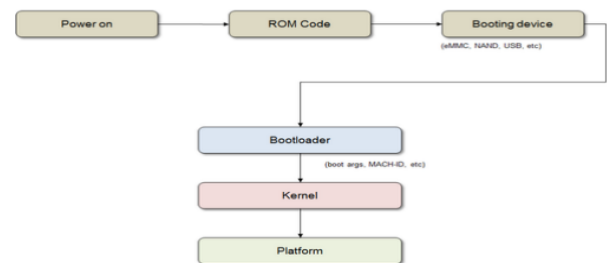
`#!/bin/bash`

Meaning: Use the bash located at /bin/bash

System Boot

Definition: System boot is the process of starting a computer and loading the operating system into main memory so that the system can begin operation.

Operating systems must be made available to hardware, so hardware can start it.



Sometimes system boot happens in a two-step process:

- ROM code loads the boot block from a fixed location.
- The boot block loads the bootstrap program from the disk.
- The bootstrap loader then loads the operating system into memory and execution continues.

Bootstrap loader: It is a small piece of code located in the ROM. It locates the kernel, loads it into the memory, and starts execution.

Common bootstrap loader

- called GRUB- Grand Unified Bootloader
- allows selection of kernel from multiple discs, versions, and options (the list of OS options you see when restarting a device. That's grub giving you options)
- loads the kernel and starts it

Working:

1. When power is turned on, the CPU initializes itself
2. CPU scans the system's ROM BIOS. Here, it finds the first instruction in the startup program
3. It instructs system to run POST (Power on self test)
4. POST first checks the BIOS chip and CMOS RAM. When no battery failure is detected, it continues initialization.
5. POST also checks all the hardware and storage devices.
POST does this by sending a command to all these devices. The devices then run an internal diagnostic.
6. After POST has concluded its checks, the BIOS starts the bootstrap process from the bootable drive.

Usually, bootstrap is located in the BIOS ROM and is not easily changeable. If a tiny bootstrap loader is located in the ROM instead, the entire bootstrap program can be edited.

- The bootstrap program is stored in a partition called boot block.
- This boot block is on a disk called boot disk.
- The bootstrap program is loaded in memory and starts executing.

Init

- Last step of booting process
- Init is the first user-level process that a kernel runs. It initializes the system, runs startup scripts, manages services, and controls runlevels until shutdown.
- It decides the run level. A run level is a mode of operation in a system that defines which services and processes will run after boot.
- The initial stage of an OS is decided by the run level.

Run levels in init

Following are the run levels of Operating System:

- 0 - System Halt
- 1 - Single user mode
- 2 - Multiuser, without NFS (Network File System)
- 3 - Full multiuser mode
- 4 - Unused
- 5 - Full multiuser mode with network and X display manager (X11)
- 6 - Reboot

Default run level to be set, would be either 3 or 5

What each does:

- 0 – shuts the system down

- 1 – only root user, no network, used for maintenance
- 2 – multiple users but no network file sharing
- 3 – full normal operation, no GUI
- 4 – not used
- 5 – full normal operation with GUI and network
- 6 – reboots the system

After init,

- Start up various daemons that support networking and other services. A daemon is a process that runs unattended in the background at all times.

Boot failure error is encountered when a system cannot boot. It indicates that the computer is not passing POST or that the devices did not pass diagnostics.

Boot

Cold boot: To start a computer that is turned off

Warm boot: To restart a computer after it's been turned on. Shortcut: ctrl+alt+del

S.NO.	COMPARISON	COLD BOOTING	WARM BOOTING
1.	Initialized by	Power button.	Reset button or by pressing Ctrl+Alt+Del simultaneously.
2.	Performed	Frequent basis.	Not very common.
3.	Alternate names	Hard Booting, Cold start and dead start.	Soft Booting.
4.	POST (Power On Self Test)	Included.	Not included.
5.	Basic	Turning a computer ON from a powerless state.	Resetting a computer from already in running state.
6.	Consequence	Does not affect the data or other hardware.	Can severely affect the system causing the data loss.

System calls

Definition: A system call is the mechanism through which a user program requests services from the OS.

Why

User programs cannot access hardware or protected resources directly. Therefore, they request the kernel to perform operations using system calls.

A system call is needed for:

1. Creation or deletion of files.
2. Reading from or writing to files.
3. Creation and management of processes.
4. Access to hardware devices such as printers or scanners.
5. Network communication such as sending or receiving packets (small chunk of data).

Working

1. A program executes normally in user mode.
2. When it needs an OS service, it makes a system call.
3. The system switches from user mode to kernel mode.
4. The kernel executes the requested operation.
5. After completion, control returns to user mode.
6. The program continues execution.

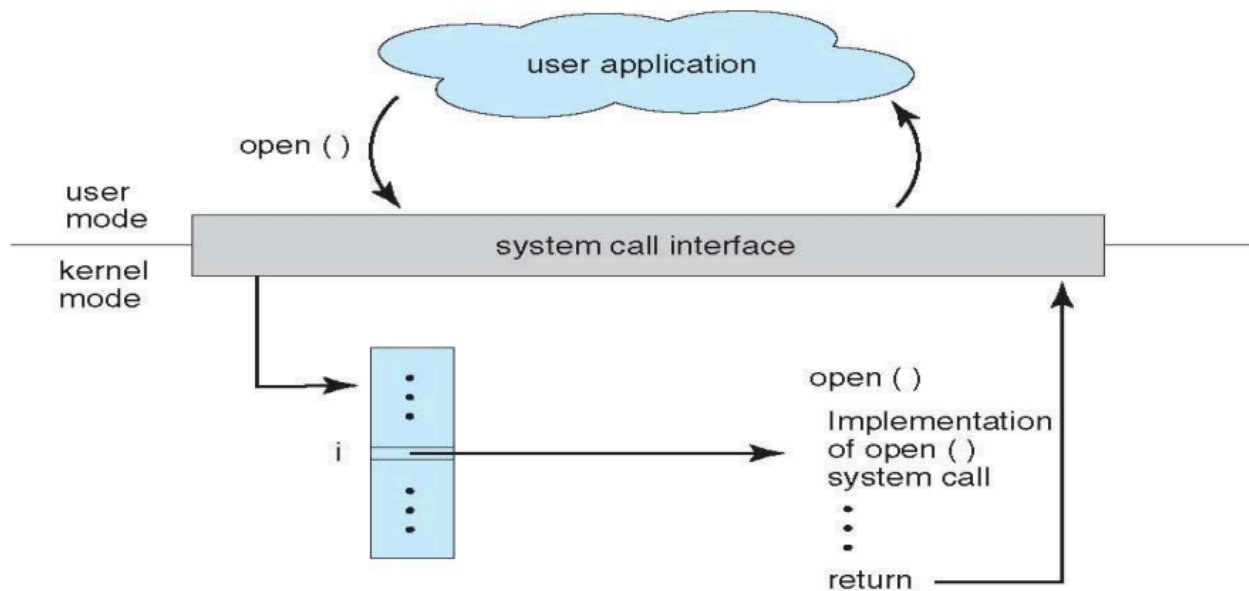
System calls are usually implemented using assembly language instructions. Some systems allow system calls to be made directly from high-level languages. Languages such as C, C++, and Perl are commonly used for system programming.

A number is associated with each system call. A system-call interface maintains a Table indexed according to these numbers. This system call interface invokes the system call in the kernel and returns output / status of system call.

Common APIs

1. Win 32
2. POSIX api
3. Java API

Api x System call relationship



System Call Parameter Passing

System calls often need extra information besides the call name. This information is called parameters. The type and number of parameters depend on the operating system and the system call.

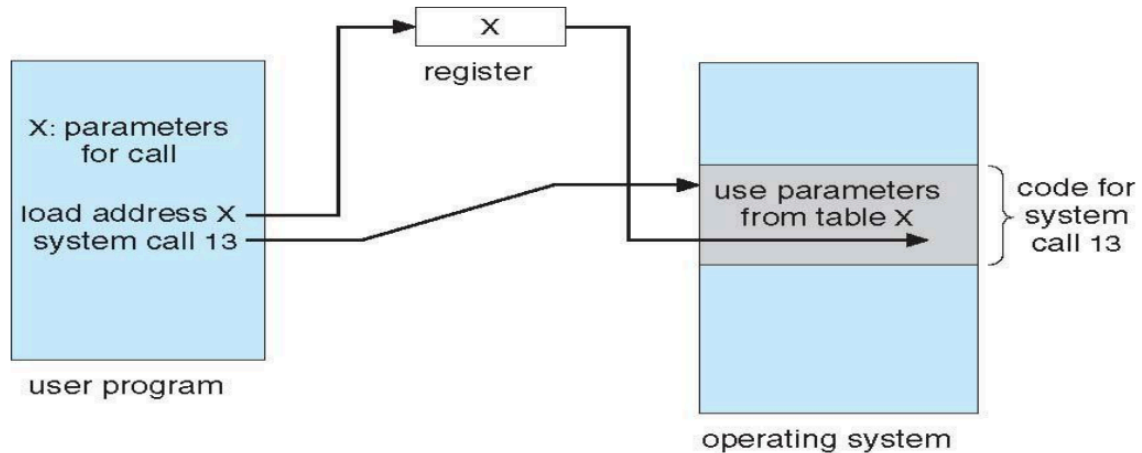
Methods of passing parameters:

1. **In register:** The parameters are placed in CPU registers before the system call is executed. This method is fast but limited because registers are few.
2. **In a memory block or table:** The parameters are stored in a block of memory, and the address of this block is passed to the operating system
3. **Stack:** Parameters are pushed onto the stack by the program, and the operating system pops them from the stack during execution.

Different operating systems use different parameter passing methods.

Parameter passing thru tables:

The program stores all its parameters in a block/table in memory, calls it X. Then it loads the address of X into a register and makes the system call. The OS looks at that register, finds the address, goes to that location in memory and reads all the parameters from table X



Types of System Calls

- Process control
- File management
- Information management
- Device management
- Communication
- Protection

Please find Iliana DCruz Promptly

Process Control

Process control system calls manage the creation, execution, and termination of processes in an operating system. They help control how processes run and interact with system resources.

Functions:

1. Create process
2. Terminate process
3. Load and execute process
4. get/set process attributes
5. Wait for time
6. Wait event/signal event
7. Allocate and free memory
8. Debug

File management

These manage the file operations.

Functions:

1. Create and delete file
2. Open and close file
3. Read, write, reposition files
4. Get and set file attributes.

Device management

Device management system calls are used by the operating system to control and interact with hardware devices such as printers, disks, keyboards, and scanners

1. request/release device
2. Read, write, or reposition
3. Get or set device attributes
4. Logically attach/detach devices. This implies connecting or disconnecting a device from a process.

Information maintenance

Information maintenance system calls are used to get or modify system and process information maintained by the operating system.

1. Gets time or date
2. Sets time or date
3. Gets and sets system data
4. Gets and sets process, file, and device attributes

Communication

Communication system calls allow processes to exchange data and coordinate with each other.

1. Creates and deletes communication connections
2. Sends and receives messages
3. Processes communicate by sending messages using a process name or host name
4. Transfers status information
5. Attaches and detaches remote devices. Plugging in or unplugging devices that aren't physically on your machine but accessed over a network.

Protection

Manages all the security related processes

1. Controls access to resources
2. Gets and sets permissions
3. Allows and denies user access

Windows	
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()

Module 2.1

Process

Process management is an important task of the operating system. The OS must

1. Interleave multiple processes' execution
2. Allocate resources to processes
3. Protect resources between processes
4. Let processes share and exchange information
5. Enable process synchronization

Process

Definition:

A program currently executing in memory is called a process. It is characterised by a sequence of instructions, a current state, and an associated set of system resources.

It comprises:

1. Program code
2. Set of data
3. Number of attributes

Process Attributes

These are stored in the PCB (Process control block)

1. Identifier: Helps to uniquely identify processes
2. State: Flag that tells if a process is running, etc.
3. Priority: What is the priority level of this process relative to the rest
4. PC: Program counter holds the next executable instruction's address
5. Memory pointers: Points to program code and associated data.
6. Context data: Data in processor registers during the execution of the process
7. IO status information: information about the pending io operations, io requests, any io devices assigned
8. Accounting information: Stores CPU time used, total elapsed time, and maximum allowed runtime of a process.

pimscapi

Process Control Block

- Data structure to hold process attributes

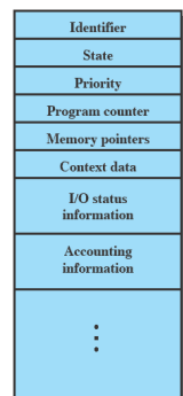


Figure 3.1 Simplified Process Control Block

- Created and managed by the OS
- A pcb has just enough information to continue a process if it had been previously interrupted.
- Enables the OS to support multiprocessing
- PCB+ associated data + program code = process

Trace of a Process

- A list of sequence of instructions executed on a process that helps characterize the behavior of this process is called a trace.
- Behaviour is characterized by showing how traces of different processes are interleaved.
- CPU switches between processes quickly. So traces appear happening simultaneously.

Process Execution

Consider three processes are being executed and are all in the main memory. We also have a dispatcher that switches the processor from one process to another.

The trace of processes is:

5000	8000	12000
5001	8001	12001
5002	8002	12002
5003	8003	12003
5004		12004
5005		12005
5006		12006
5007		12007
5008		12008
5009		12009
5010		12010
5011		12011

(a) Trace of Process A (b) Trace of Process B (c) Trace of Process C

And the processor's functioning on these processes is: (check img)

Note that the 100 to 105 addresses are the addresses of the dispatcher. It is here when the dispatcher switches CPU from one processor to another.

1	5000		
2	5001		
3	5002		
4	5003		
5	5004		
6	5005		
		-----	Timeout
7	100		
8	101		
9	102		
10	103		
11	104		
12	105		
13	8000		
14	8001		
15	8002		
16	8003		
		-----	I/O Request
17	100		
18	101		
19	102		
20	103		
21	104		
22	105		
23	12000		
24	12001		
25	12002		
26	12003		
27	12004		
28	12005		
		-----	Timeout
29	100		
30	101		
31	102		
32	103		
33	104		
34	105		
35	5006		
36	5007		
37	5008		
38	5009		
39	5010		
40	5011		
		-----	Timeout
41	100		
42	101		
43	102		
44	103		
45	104		
46	105		
47	12006		
48	12007		
49	12008		
50	12009		
51	12010		
52	12011		
		-----	Timeout

Process creation

When a new process is to be added, it is added to the set of processes currently under execution.

OS creates a process control block and allocates address space in the main memory. This creates a new process.

There are three triggers for process creation –

- 1. New job in batch system** – someone submits a job (program + its data) to be run, OS creates a process for it.
- 2. User logs in** – user opens a terminal, OS spins up a shell process for them to interact with.
- 3. Parent spawns a child** – a running process creates another process. The creator is the parent, the new one is the child. Like a shell process creating a process to run a command you typed.

Process termination

Events leading to process termination

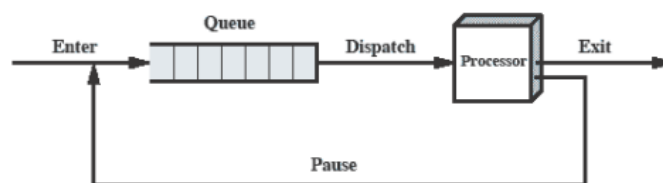
1. User specified: Logout/termination of application
2. Normal completion: Process was completed and terminated
3. Time limit exceeded TLE: Process ran for too long
4. Memory unavailable: Process required too much memory

5. Bounds violation: Process tried accessing memory it is forbidden to access
6. Protection error: Attempts to use a resource that is forbidden to access (Trying to write in a read only file for example)
7. Arithmetic error: Example: division by zero
8. Time overrun: Process waited too long for a dependency to complete
9. IO failure: Process waited too long for a required IO operation to complete
10. Invalid Instruction: Process tried to execute an invalid instruction
11. Privileged Instruction: Process tried to execute an instruction that is reserved only for the OS
12. Data misuse: data of wrong type or not initialized data
13. OS intervention: OS terminates the process (e.g. in case of deadlock)
Deadlock is when two or more processes are all waiting on each other's completion and no process can move any further.
14. Parent termination/parent request: Parent process has authority to terminate any of its offsprings (child processes)

Two state process model

A process can be in two states

1. Running
 2. Not running
- When a new process is created the OS enters the process into the system in Not-running state. A PCB is created and the OS knows that a process exists and is waiting to execute.
 - Occasionally, the running process will be interrupted and the dispatcher sends some other process to run.
 - Process in the running state moves to the not running state
 - One of the processes in the not running state moves to the running state.
 - Not running processes are kept in a single queue. Each entry points to the PCB.



(b) Queuing diagram

Five state process model

Why needed: 2 state process models are inadequate. It would have been effective only if all processes were always ready to execute.

What if some processes in not-running state are ready to execute, while others are blocked; waiting for completion of I/O operation.

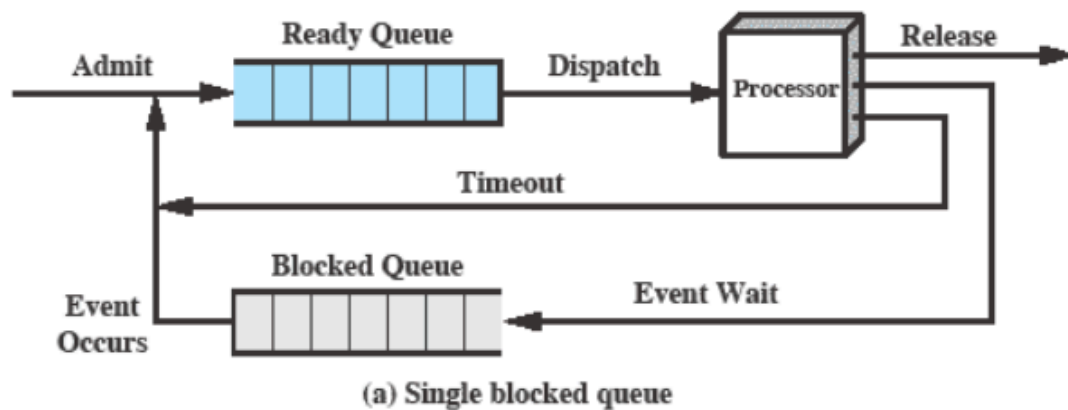
Dispatchers should first scan the queue looking for an unblocked process that has been in the queue for the longest time period.

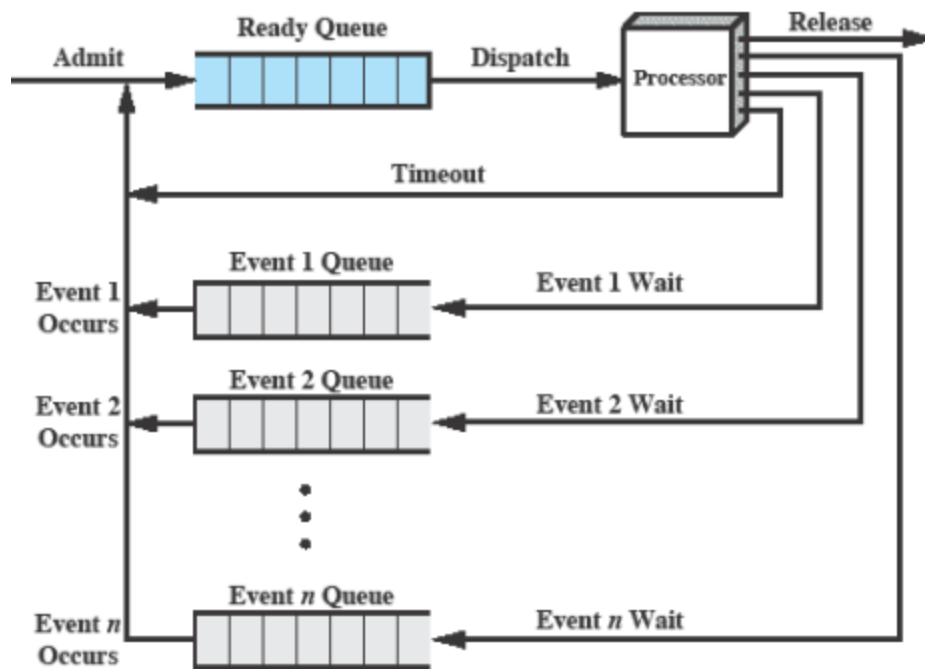
We split the not-running state further: Blocked and Ready



The 5 states:

1. Running
2. Ready: Process ready for execution
3. Blocked: Process waiting on some action/event
4. New: Newly created process that is not yet loaded in main memory
5. Exit: Process released from the queue because it has terminated or been aborted.





(b) Multiple blocked queues

Suspended process

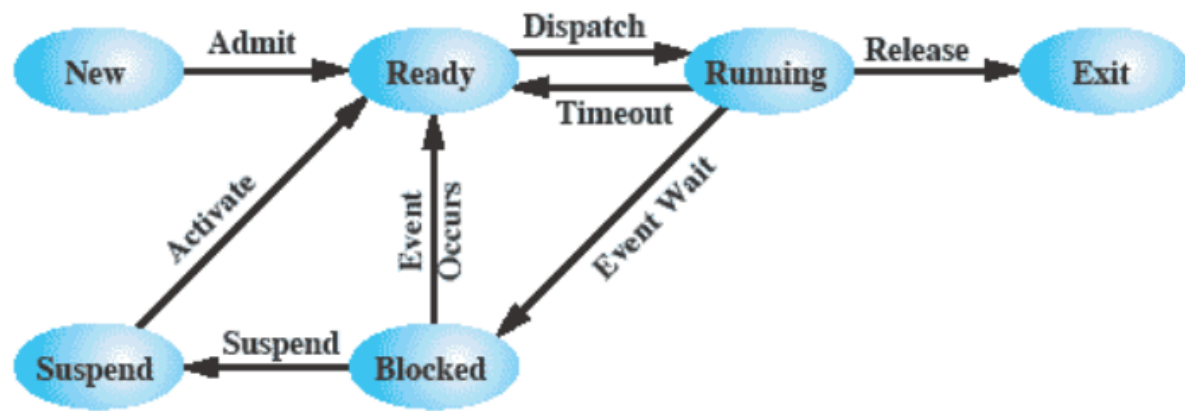
A process must reside in main memory to execute. The processor is much faster than I/O devices, so many processes may end up waiting for I/O operations. When this happens, the CPU may remain idle because all processes in memory are blocked.

- One solution is to expand main memory so more processes can stay in memory at the same time.
- Another solution is swapping. Swapping moves part or all of a process from main memory to disk, freeing memory space.

If no process in main memory is in the ready state, the OS swaps blocked processes out to disk and places them in a **suspend queue**.

The suspend queue contains processes that have been temporarily moved out of memory. The freed memory is then used to bring in another process, which could be a newly admitted process or a previously suspended one.

Process Model: One suspend State



(a) With One Suspend State

A process is suspended when:

1. It is waiting on an event
2. The process has been swapped out of main memory. (Sometimes a process is swapped when memory is full, too)

Two new states emerge:

1. Blocked/Suspend: Process is waiting for an event and is in the secondary memory
2. Ready/Suspend: Process is ready to execute upon loading in main memory. It is currently in secondary memory.



(b) With Two Suspend States

Suspended Process State Transitions

Suspended process state transitions describe how processes move between memory and disk.

- A blocked process may be swapped out to disk, causing a Blocked → Blocked/Suspend transition.
- When the event it was waiting for occurs while it is still on disk, and it becomes ready but remains suspended, giving Blocked/Suspend → Ready/Suspend.
- When the OS brings this process back into main memory, it becomes Ready, giving Ready/Suspend → Ready.
- A ready process in memory may also be swapped out to free memory, producing Ready → Ready/Suspend.
- A new process may either be placed directly in the ready state (New → Ready)
- Or it could be kept on disk if memory is insufficient (New → Ready/Suspend).
- A suspended blocked process may be brought back to memory but still remain blocked (Blocked/Suspend → Blocked).
- A running process that is preempted/on timeout may be directly swapped to disk, giving Running → Ready/Suspend.
- A process may terminate from any state, resulting in a transition to Exit. Any → Exit

Why Must a Process be Suspended

Reason	Comment
Swapping	The OS needs to release sufficient main memory to bring in a process that is ready to execute.
Other OS Reason	OS may suspend: background utility process, or process suspected of causing a problem.
Interactive User Request	e.g. debugging or in connection with the use of a resource.
Timing	A process may be executed periodically (e.g., an accounting or system monitoring process) and may be suspended while waiting for the next interval.
Parent Process Request	A parent process may wish to suspend execution of a descendent to examine or modify or to coordinate the activity of various descendants.

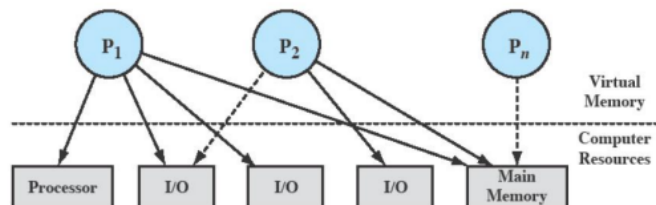
Characteristics of Suspended Process

1. Not available for execution immediately
2. May/may not be waiting for an event
3. If a blocked-suspend process was waiting for an event and the event occurs, it still cannot execute because it is not in main memory
4. It was placed in suspended state by an agent
5. Process is not removed from suspend state until the agent orders removal

OS control structure

During execution, each process needs some resources. These resources could include IO, memory, processor, etc.

The OS is responsible for managing the use of system resources.



To control processes and manage resources, the OS needs:

1. Current status of processes and resources
2. OS maintains tables of information about each entity. Memory table, IO table, file table, and process table.

Memory table

- Keeps track of main and secondary memory
- Some part of the main memory is reserved for the OS, the remaining is allocated to processes
- Secondary memory holds processes that were swapped out
- Memory table must track:
 - Main memory allocation – which process owns which chunk of RAM
 - Secondary memory allocation – which process is swapped where on disk
 - Protection attributes – who's allowed to access which memory region
 - Virtual memory info – mappings between virtual and physical addresses

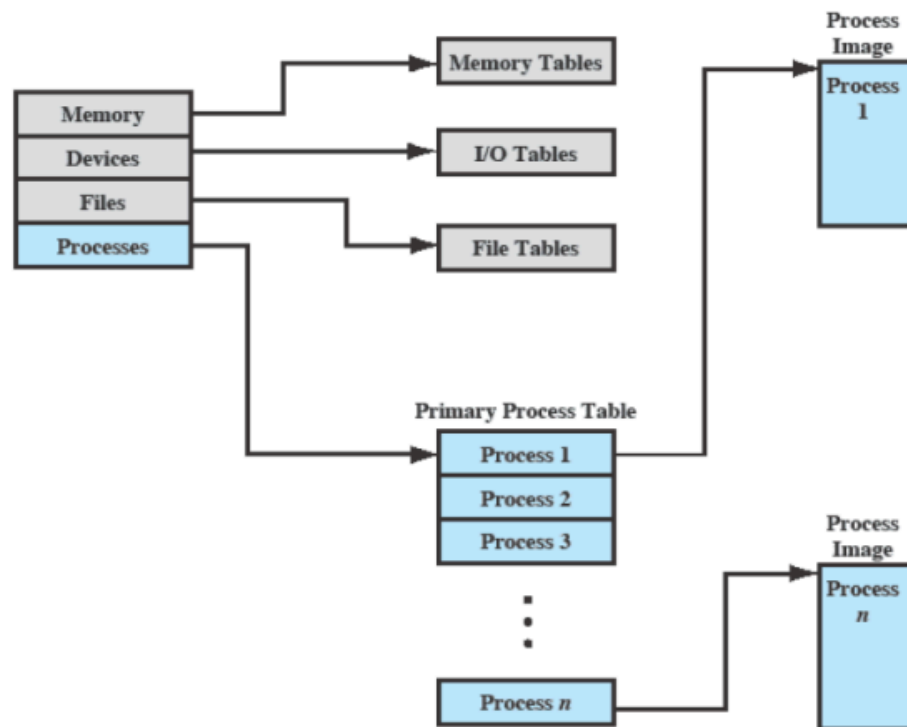
IO table

- Manages IO devices
- Tells if an IO device is available
- Tells the status of an IO operation
- It records the memory address where data is read from or written to during the I/O operation.
- Records the source and destination of an IO operation.

File Table

- Provides information on
 - Existence of files
 - Their location
 - Current status
 - Other attributes
- Maintained by file management systems sometimes

OS Control Tables: General structure



Process Control Structure

To control a process, the OS must know:

1. Location of process
2. State
3. Attributes

Collection of program, data, stack, and attributes is called **process image**.

A process image is maintained as a contiguous block in secondary memory. Modern OS use paging hardware so that we don't need contiguous memory. Some pages of the process are kept in main memory, others stay in secondary and brought to main as needed.

To execute a process, the entire process image must be loaded in the main/secondary memory. Only the pages currently needed must be in main memory for execution.

The primary process table is the OS data structure that stores an entry for every process in the system, usually pointing to that process's PCB. The **process table** must show the location of each page of the process image.

Elements in process image include:

- User program: Program to execute

- System stack: LIFO stack to store parameters
- PCB

OS keeps a process table → each entry points to a PCB → PCB + program + stack = process image → only needed pages stay in RAM, rest on disk.

Process Attributes

Information required by OS for managing each process can be grouped into three general categories:

1. Process Identification information
2. Processor State information
3. Process Control information

Process Identification Information

Each process has a unique identifier.

1. PID: Identifier of process
2. PPID: Identifier of the parent of the process
3. User identifier: the ID of the user who owns or started the process, used by the OS to link the process with that user's account and permissions.

Processor State Information

It consists of the content of processor registers.

1. **User visible registers:** Referenced by machine language, directly used by program
2. **Control and status registers:** Processor registers that control its operation. Includes Program counter, condition codes, flag registers
3. **Stack pointers:** Stores parameters and calling addresses. SP → Top of flag

Process Control information

Mnemonic: sdc prom

Scheduling and state information:

1. It includes the **process state** such as running, ready, or blocked.
2. The PCB also stores **priority information** like default priority, current priority, and the maximum priority allowed.
3. **Scheduling** related information such as the scheduling algorithm used, waiting time, and execution time.
4. The event field stores the **event** the process is waiting for if it is blocked.

Data structuring information:

- Links processes together in structures like queues, rings, or other lists used by the scheduler.
- For example, processes with the same priority may be linked in a ready queue.
- The PCB therefore contains pointers to other processes to support these scheduling structures.

Inter-process Communication: Flags, signals, and messages are used to communicate between two independent processes.

Process privileges: This attribute tells what memory locations may be accessed

Memory Management: This attribute is a pointer to segment and/or page tables that describe virtual memory assigned to process

Resource Ownership and Utilization: Talks about which resources are controlled by the process. Holds the history of utilization of processor and other required resources.

Structure of Process Images in Virtual Memory

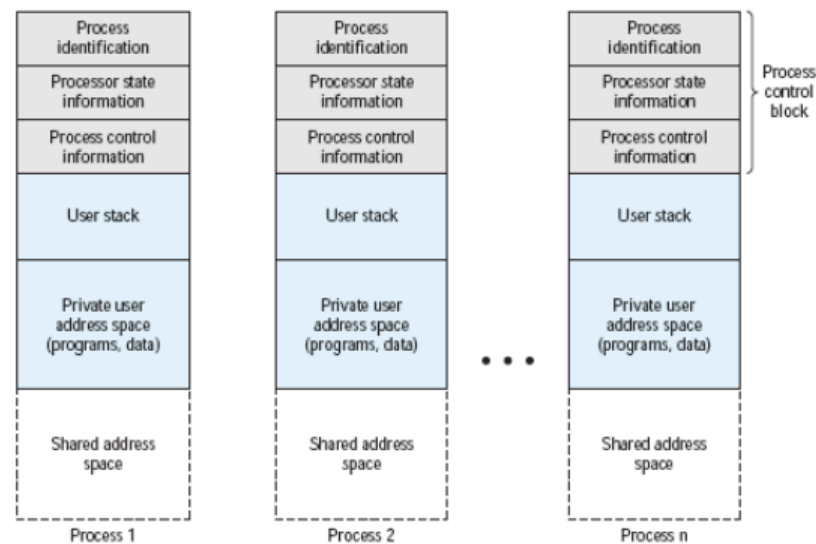


Figure 3.13 User Processes in Virtual Memory

Role of PCB

- It is the most important data structure in an OS
- It contains all information about the process that an OS may need
Collectively, all PCBs tell you exactly what every process is doing at any moment – which IS the state of the OS.
- PCB must be protected
 - Faulty routine might damage it, preventing OS from managing process
 - Any design change affects multiple OS modules

Modes of Execution of Process

The mode in which the CPU executes the process's instructions.

Most processors have 2 modes

1. User mode: Less privileged, user programs execute here
2. Kernel Mode: More privileged, instructions are executed with privilege.
Also called system/control/protected mode.

A flag bit in the PSW (A register called Program Status Word) tells the mode of execution to the processor.

When a user makes a call for an OS request or when an interrupt triggers the OS routine, the mode is set to kernel mode. Upon return from service, mode returns to user mode.

Kernel Functions

Kernel is the core part of an operating system that manages hardware, memory, processes, and system calls, running in privileged kernel mode.

1. Process management
2. Memory management
3. IO management
4. Support

simp

Process Management

Process creation

- The OS first assigns a unique process identifier (PID) to the new process.
- It then creates an entry for the process in the primary process table.
- The OS allocates memory space for the process, including space for the process image.
- After that the process control block/PCB is initialized.

- Most state information is set to zero, while the program counter and stack pointer are initialized appropriately.
- The OS then sets up the required linkages by inserting the process into scheduling structures such as ready queues that are implemented as linked lists.
- Finally the OS creates or updates other supporting data structures, such as accounting information maintained for the process.

Process switching

When a running process is interrupted and the OS assigns another process to running state, it is called process switching

A process switch will occur when:

Mechanism	Cause	Use
Interrupt (Ex. Clock interrupt, I/O interrupt, Memory Fault)	External to and independent of execution of the current instruction	Reaction to an asynchronous external event
Trap	Associated with the execution of the current instruction	Handling of an error or an exception condition
Supervisor call	Explicit request (system call)	Call to an operating system function

Steps in a full process switch:

1. Save context of processor, including program counter and other registers
2. Update PCB: change state from running -> whatever
3. Move PCB to appropriate queue
4. Select new process to execute
5. Update PCB of new process: whatever -> running
6. Update memory management data structures
7. Restore context of the selected process

Note: Occurrence of interrupt not necessarily means a process switch. It can also happen due to mode switching. We save the processor state information when interrupt occurs and restore it after interrupt handler has executed, and control is returned to the program.

Mode Switching

This occurs without changing the state of the process that is currently in the Running state.

Mode switching is when the CPU changes its execution mode between user mode and kernel mode. When a user program requests an OS service (system call) or

an interrupt occurs, the CPU switches from user mode to kernel mode so the OS can run privileged instructions. After the OS finishes the service or interrupt handling, the CPU switches back to user mode and the user process continues execution.

Module 2.2

Scheduling

Processor scheduling

The OS allocates CPU execution time among competing processes. The CPU provides execution time as the resource. This time is assigned using a scheduling policy. The goal is to decide which process runs and when. Scheduling aims to meet system objectives such as good response time, high throughput, and efficient CPU utilization.

Objectives

1. Fairly share processor time
2. Prevent process starvation
3. Efficient use of processor
4. Have low overhead
5. Prioritize processes when needed

Types

1. **Long-term** – decides which new process joins the active set (FCFS, priority, etc.)
2. **Medium-term** – decides which swapped-out process comes back into RAM
3. **Short-term (Dispatcher)** – decides which ready process gets the CPU next; triggered by interrupts/system calls
4. **I/O scheduling** – decides which process's I/O request is served first when I/O becomes free

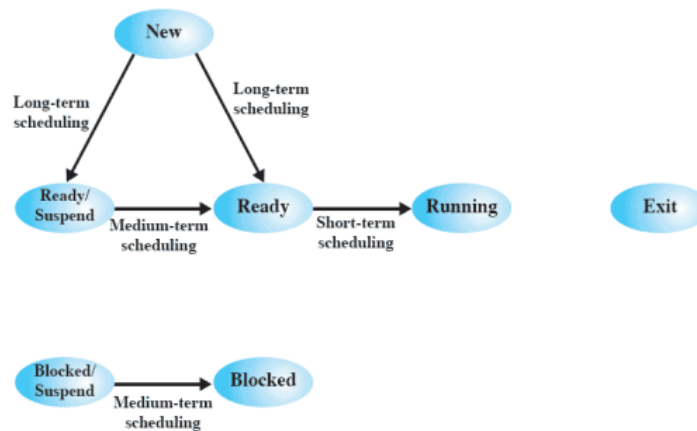


Figure 9.1 Scheduling and Process State Transitions

Short time scheduling: User vs System

When process goes from ready to running, based on whose best interests is it being forwarded?

User oriented:

1. Related to system behaviour perceived by user
2. It focuses on user-oriented goals such as fast response time and service provided to user.

System oriented

1. Efficient utilization of processor
2. Focus on system performance

Short term scheduling may also be based on performance. If it is performance related, we could decide based on throughput and response time. If it is not performance based, we could decide by any qualitative means.

It could also be divided into non-preemptive and preemptive.

Non preemptive sts implies that once a process begins, it will continue till it's blocked/terminated. Example: FCFS

Preemptive sts implies that the currently running process may be interrupted and moved to the ready queue by the OS. preemption can occur both when triggered or automatically. Example: Round Robin

Interdependent Scheduling

User Oriented Performance Related

- Turnaround time: Completion Time - Arrival Time
- Normalized turnaround time = turnaround time / burst time.
- Response time is the time from submitting a request until the first response appears.
- Deadlines mean processes must finish before a specified time.

User Oriented

- Predictability means similar jobs should take roughly the same time even under different system loads.

System oriented

- Throughput is the number of processes completed per unit time.
- Processor utilization is the percentage of time the CPU is busy. A high percentage indicates efficient processor usage.

Other system criteria

- Fairness means processes are treated equally and none starve.
- Enforcing priorities favors higher priority processes.
- Balancing resources keeps system resources efficiently used.

Priority Scheduling

Implies choosing a high priority process over a low priority one. To implement, we have multiple ready queues to represent each level of priority.

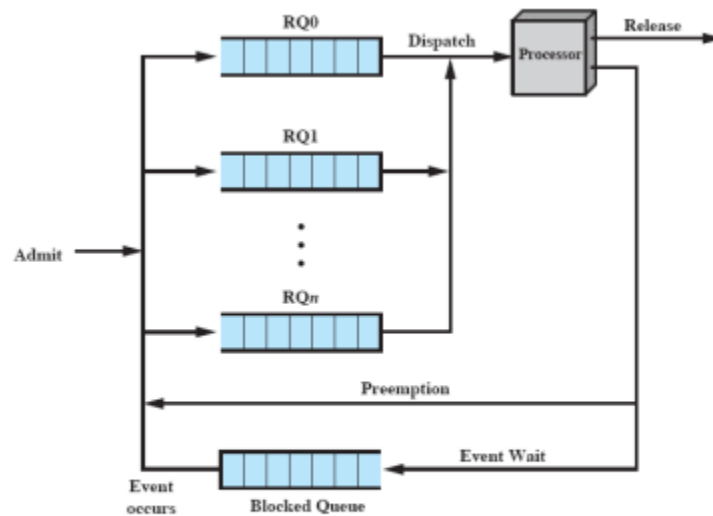


Figure 9.4 Priority Queuing

Problem:

Starvation: Low priority processes may starve if there is a steady supply of high priority processes

Solution:

A process's priority can be made dynamic based on its age in the queue.

FCFS

1. Average wait time is long
2. Short process behind long process leads to convoy effect. The Convoy Effect is a phenomenon where a large or resource-intensive process ties up system resources and causes a backlog of other processes waiting to use those same resources.
3. Whole OS slows down due to a few large processes.

Round Robin

1. Uses clock based preemption
2. Each process is given a quantum of time before being preempted. Called time slicing
3. Small quantum: More context switches
4. Large quantum: RR becomes FCFS
5. Limitations:
Processor bound processes(processes that spend most of their time doing computation, rarely waiting for I/O.) receive unfair portion of time
Leads to inefficient IO use
Solution: Virtual RR

Virtual RR

1. After I/O, processes enter the auxiliary queue, get priority over ready queue, and run only remaining time from their original time quantum.

SPN: Shortest Process Next

1. Non preemptive
2. Possibility of starvation for long process
3. Low predictability of long process

STRN

1. Optimal when AT is different
2. Turnaround time = Completion - Arrival
3. Waiting Time = Turn Around Time - Service Time
4. Normalized Turn Around Time = Turn Around Time / Service Time

Feedback scheduling/multilevel feedback queue

- Scheduler penalizes long running jobs by lowering priority based on executed time
- Uses preemption with time quantum
- Dynamically adjusts priorities.

Problem

In a simple MLFQ, turnaround time of longer processes is stretched. This leads to starvation.

Remedy: Use different time quantum in queues.

For example, first ready queue= 1 unit

Second = 2 unit and so on

Module 2.3

Threads

Process has two characteristics:

1. **Resource ownership:** It owns memory, I/O devices, files, and virtual address space.
2. **Scheduling/execution:** follows an execution path through programs, may interleave with other processes, and has execution state and dispatching priority.

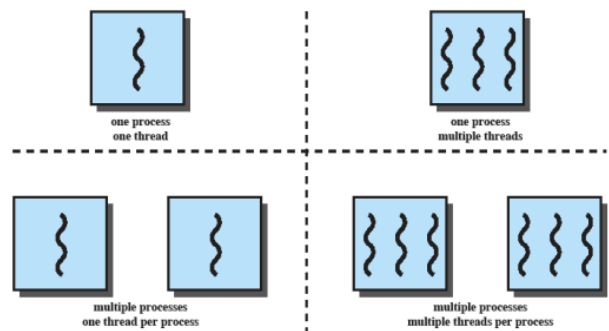
These two are treated independently by the operating system.

Thread: A thread is the smallest unit of CPU scheduling.

Multithreading: the ability of an OS to support multiple mini-processes to run within a process.

Four approaches

1. 1 process 1 thread
2. 1 process many threads
3. Many processes 1 thread
4. Many processes many threads
(linux, windows)



Threads within a process

Each thread has the following:

Mnemonic Sleepy Elephants Throw Pizza Slices Softly

1. Execution state (running, ready, etc.).
2. Saved thread context when not running.
3. Independent program counter within process.
4. Execution stack.
5. Per-thread local variable storage
6. Shared process memory and resources.

Thread vs Process

Single-threaded process model (Process)

1. No separate thread concept.
2. Process representation includes PCB.
3. Includes user address space.

4. Includes user and kernel stacks.
5. The running process controls CPU registers.
6. Registers saved when the process is not running.

Multithreaded environment (Thread)

1. One PCB and one address space per process.
2. Each thread has a separate stack.
3. Each thread has a Thread Control Block (TCB).
4. TCB stores registers, priority, thread state.
5. Threads share process memory and resources.
6. Threads run in the same address space.
7. Data changed by one thread visible to others.
8. Threads can read the same memory concurrently.
9. File opened by one thread usable by others.

Benefits

- Less creation time: Thread can be created much faster than process
- Less termination time: Thread can be terminated faster
- Less switching time: Switching between threads is faster
- Efficiency enhancement in communication: Threads can share memory and files and can communicate faster without invoking the kernel

Use of threads

1. Spreadsheet: One thread displays menu while the other executes commands
2. Asynchronous processing: Power failure protection; write contents of RAM to disk every 100ms
3. Speed of execution: Compute one batch of data while reading next batch; multiprocessor scenario

Thread functionality

Thread states

1. Running
2. Ready
3. Blocked

Threads do not have a suspended state by itself. If a process is suspended, all its threads are suspended.

Thread Operations

1. Spawn (Create a child thread)
2. Block
3. Unblock
4. Finish

Multithreading on a Uniprocessor

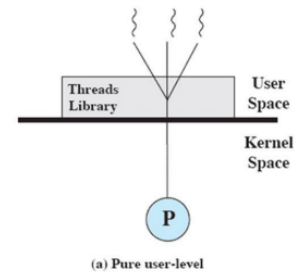
Multithreading enables interleaving of multiple threads within multiple processes. Only one thread runs at a time (one CPU), but the OS switches between threads so fast it appears parallel.

Thread implementations

- User level threads
- Kernel level threads

ULTs

- All thread management done by the application
- Kernel does not know that threads are existing



Threads library: application level functions for thread management.

1. Creates and destroys threads.
2. Handles message passing and data sharing.
3. Schedules threads.
4. Saves and restores thread context.

Thread Creation

1. Application begins with a single thread.
2. All threads belong to one process managed by the kernel.
3. Application can spawn new threads in the same process.
4. Thread spawning is done using the spawn utility in the threads library.
5. The library creates data structures for new threads.
6. Control passed to a ready thread using scheduling algorithm.
7. Current thread context saved and restored when needed.
8. All operations occur in user space.
9. The kernel schedules only the process.

If a process becomes blocked due to a system call, the kernel blocks the process. Thread library may still consider the thread running. On clock interrupt, the kernel moves the process to the ready state. The thread

library still shows the same thread running. If one thread waits for another, the waiting thread becomes blocked. Another ready thread becomes running.

Advantages

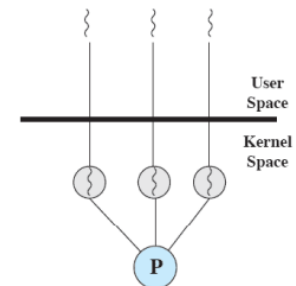
- Thread switching does not require kernel privileges.
- Thread management data stored in user space.
- Scheduling can be application specific.
- The scheduling algorithm can be customized without changing the OS scheduler.
- Works on any operating system.
- Kernel modification not required.

Disadvantages

- Cannot use multiprocessing effectively.
- If one thread blocks on system call, all threads block.
- The solution is using multiple processes, but loses thread advantage.
- Jacketing converts blocking system calls into non-blocking calls.
- Thread calls application level I/O jacket routine to check I/O availability.

KLTs

- Kernel maintains context for process and threads.
- Thread management done by kernel, not application.
- Scheduling performed on thread basis.
- Example: Microsoft Windows.



Advantages of KLT:

1. Kernel can schedule threads on multiple processors.
2. If one thread blocks, another thread can run.
3. Kernel routines can also be multithreaded.

Disadvantage of KLT:

1. Thread switching requires kernel mode switching.

	ULT	KLT
Managed by	User-level library	OS kernel

Kernel aware?	No	Yes
Context switch	Fast (no kernel involvement)	Slow (kernel call needed)
If one thread blocks	Entire process blocks	Only that thread blocks
Multiprocessor use	Can't run threads in true parallel	Can run threads on multiple CPUs
Example	Java green threads	Linux, Windows threads

Combined Thread Approach

- Threads are created and managed in user space, no kernel involvement.
- The application itself handles scheduling and synchronization.
- Many ULTs map to fewer KLTs (many-to-few mapping).
- Threads can run on multiple processors simultaneously.
- If one thread makes a blocking call, only that thread blocks – rest continue.

Example: Solaris

Relationship Between Thread and Processes

Threads:Processes	Description	Example Systems
1:1	Each thread of execution is a unique process with its own address space and resources.	Traditional UNIX implementations
M:1	A process defines an address space and dynamic resource ownership. Multiple threads may be created and executed within that process.	Windows NT, Solaris, Linux, OS/2, OS/390, MACH
1:M	A thread may migrate from one process environment to another. This allows a thread to be easily moved among distinct systems.	Ra (Clouds), Emerald
M:N	Combines attributes of M:1 and 1:M cases.	TRIX

Symmetric Multiprocessing

Symmetric multiprocessing: multiple processors share memory and OS; each processor runs tasks equally. Kernel can execute on any processor. Each processor self-schedules from the pool of available processes/threads.

This complicates the OS as two processors should not choose the same process and that processes are not lost from queue.

Some issues for designing include memory management, reliability, and scheduling, etc.

Concurrent processes

Multiple processes or threads execute at the same time on different processors. They may access the same data or resources. This can cause race conditions, inconsistency, conflicts.

1. Kernel functions need to allow several processors to run the same kernel code simultaneously
2. Kernel tables and management structures need to be managed with greater care to avoid deadlocks.

Scheduling

Any processor can schedule, so we need to avoid conflicts. If kernel level multithreading is implied, then we can schedule multiple threads from the same process at the same time on different processors.

Three issues are seen:

- a. Assignment of processes to processors. Deciding which CPU gets which process/thread.
- b. Use of multiprocessing on one processor. Deciding how many processes stay ready on each CPU.
- c. Actual dispatching of process. Actually giving CPU control to a selected process/thread.

Solution

1. Dynamic Assignment: threads are moved from a queue for one processor to a queue for another processor. If one CPU is overloaded and another is free, work is shifted.
2. Static Assignment: Permanently assign of process/thread to a processor. May result in processors remaining idle; while others have a backlog/pending work, so use a common queue. All processes go to a global queue and are scheduled to any available processor.
3. Two methods further

a. master/slave

Master-slave architecture is a multiprocessor design where one processor (master) controls scheduling, I/O, and OS functions. Other processors (slaves) execute user processes assigned by the master. The master manages system resources and coordination, simplifying control but creating a possible bottleneck if the master processor becomes overloaded. If master fails, everything breaks down

b. Peer to peer

Peer-to-peer architecture is a multiprocessor design where all processors are equal. Each processor runs the operating system and performs scheduling and I/O. Processes can run on any processor. Workload is shared among processors, improving parallelism and avoiding the bottleneck found in master-slave systems.

Synchronization

With multiple active processes having potential access to shared address spaces or shared I/O resources, care must be taken to provide effective synchronization

A common synchronization mechanism is used in multiprocessor OS.

Memory management

Multiple processors accessing shared memory can cause inconsistency in pages or segments, inefficient use of parallel memory hardware, and conflicts in paging or page replacement

1. OS coordinates paging across processors
2. OS uses hardware parallelism (e.g., multiported memory) for performance.

Reliability and Fault tolerance

1. Schedulers and other parts must recognize the loss of a processor incase it fails and restructure the management tables as needed to keep the system working.

Thread Scheduling

Threads execute separate from the rest of the process. An application can have many threads. They run together in the same memory. On multiprocessor systems they run in parallel, giving better performance than uniprocessor systems.

Four approaches:

1. Load sharing
2. Gang scheduling
3. Dedicated processor assignment
4. Dynamic scheduling

Load sharing

Load sharing means distributing processes or threads among processors so that workload is balanced and no processor remains idle.

1. Global queue maintained
2. Each processor, when idle, selects a thread from the queue

Advantages

1. Simple
2. Evenly distributed load
3. No centralized scheduler needed
4. The global queue (of tasks) can be organized and accessed using any scheduling algorithm

Examples: FCFS, Smallest number of threads first, Pre-emptive Smallest Number of Threads First

Disadvantages

1. Can lead to bottlenecks
2. High overhead due to frequent task migration.
3. Preemptive threads are unlikely to resume execution on the same processor-> makes caching inefficient
4. If all threads are in the global queue, all threads of a given program will not gain access to the processors at the same time

Gang scheduling

Gang scheduling means scheduling a group of related threads of the same application to run at the same time on different processors. It improves coordination and communication between threads running in parallel.

1. Group of related threads are scheduled as a unit
2. All members of gang run simultaneously on different processors
3. All members start and end their time slices together

Dedicated Processor Assignment

Dedicated processor assignment means giving specific processors exclusively to a process or its threads during execution. The process runs only on those processors, improving performance and reducing scheduling overhead.

Number of Processors = No. of threads in the program for the duration of program execution

Disadvantages:

1. No multiprogramming
2. Some processors remain idle

Dynamic Scheduling

Dynamic scheduling means the operating system assigns processors to threads during execution based on current load and availability, improving processor utilization and system performance.

Dynamic Scheduling allows the OS to adjust the load to improve utilization/performance.

Disadvantages of Dynamic Scheduling:

- High overhead due to frequent scheduling decisions
- Threads/processes may migrate often between CPUs
- Cache inefficiency due to migration
- Load-balancing logic increases complexity
- Can cause bottlenecks if common queue is used
- Unpredictable execution order sometimes reduces performance

Module 3.1,3.2,3.4

Process Synchronization

Processes can execute simultaneously. They can be interrupted, and partially complete before finally executing.

- Concurrent access to shared data results in data inconsistency.
- Data should always stay correct and consistent. We use control methods to run processes in order so that data inconsistency is avoided.

Producer-consumer problem

A producer process produces information that is consumed by a consumer process.

In general practice, processes can execute concurrently and may be interrupted at any time. When multiple processes share data without coordination, the result can be inconsistent; this is where synchronization becomes necessary. So that the consumer does not try to consume an item that has not yet been produced.

Two types of buffers:

1. Unbounded
 - a. Places no practical limit on buffer size
 - b. Consumer might have to wait for a few items
 - c. Producer can always produce items
2. Bounded
 - a. There is a fixed buffer size
 - b. When the size is full capacity, the producer cannot make more information.
 - c. When buffer is empty, consumer cannot consume anymore
 - d. IN points to the next free position in the buffer; OUT points to the first full position in the buffer. The buffer is empty when $in == out$; think about circular queue and front and rear.

The bounded buffer (producer-consumer) problem illustrates the issue. A shared counter variable tracks full buffers. The producer increments it (counter++) and the consumer decrements it (counter--). If both execute concurrently without coordination, the final value may be incorrect.

Code for producer process

```
while (true) {
    /* produce an item in nextProduced */
    while (counter == BUFFER_SIZE)
        ; /* do nothing */
    buffer[in] = nextProduced;
    in = (in + 1) % BUFFER_SIZE;
    counter++;
}
```

Code for consumer

```
while (true) {
    while (counter == 0)
        ; /* do nothing */
    nextConsumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    counter--;
    /* consume the item in nextConsumed */
}
```

What's happening in these codes?

Producer keeps making items and adding them to the buffer, but stops if the buffer is full (`counter == BUFFER_SIZE`). Consumer keeps taking items out of the buffer, but stops if the buffer is empty (`counter == 0`). Both update the same counter variable : producer increments it, consumer decrements it : which causes problems when they run at the same time (race condition).

Why `counter++` and `counter--` are not safe

These operations compile into multiple machine instructions, not one. The CPU can context-switch mid-way:

`counter++` expands to:

```
register1 = counter
register1 = register1 + 1
counter   = register1
```

`counter--` expands to:

```
register2 = counter
register2 = register2 - 1
counter   = register2
```


If the CPU interleaves these steps (e.g. producer reads, consumer reads, both write), both see a stale value and one update is lost. Starting from counter = 5, the final value could be 4, 5, or 6 depending on the order.

Race condition

A race condition occurs when multiple processes access and manipulate shared data concurrently, and the final outcome depends on the exact order of execution. The fix is to ensure only one process manipulates the shared data at a time.

The critical section problem

A critical section (CS) is the portion of code where a process accesses shared resources. For example, updating a table, writing a file, modifying shared variables. The system must ensure that when one process is inside the Critical Section, no other process can enter it.

Each process's code is split into four regions:

- Entry section: requests permission to enter the CS
- Critical section : accesses shared data
- Exit section : signals departure from the CS
- Remainder section : the rest of the process's non-critical code

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (TRUE);
```

Three requirements for a valid Critical Section (CS)

1. **Mutual exclusion:** Only one process inside the CS at a time.
2. **Progress:** If the CS is empty, only interested processes compete to enter, and the decision cannot be postponed forever.
3. **Bounded waiting:** There must be a limit on how many times other processes can enter the CS before a waiting process gets its turn.

Two approaches are used to handle critical sections in OS depending on if the kernel is preemptive or not.

- **Preemptive kernel:** Allows a process to be interrupted while running in kernel mode. This type is preferred because it is more responsive and better for real-time systems, but harder to design safely due to race condition risk. The critical section needs explicit protection.

- Non-preemptive kernel: A kernel process runs until it has finished. Then, it exits, blocks, or voluntarily releases the CPU. Nearly free from race conditions since only one kernel process is active at a time. Critical section automatically safe.

Software solutions to the CS problem

All three algorithms (algorithm 1,2,3) assume that reading or writing a single variable completes in one shot, it can't be interrupted halfway. (I.e. the single variable actions are atomic). The shared variables, however, are not protected by hardware, only by logic.

Algorithm 1 : strict alternation

Uses a shared integer turn. If $turn == i$, process P_i may enter the CS.

```
while (turn != i);    // busy wait until it's my turn. This is the entry code.
//process keeps checking if it is its turn yet.
```

```
/* critical section-> process works here. */
```

```
turn = j;              // hand turn to the other process. Exit code.
/*remainder section*/
```

When P_0 exits the cs and updates turn to j, it enters the remainder section. Now p_j can enter it. Anytime before this, p_j has to be waiting.

After P_0 comes out then P_j gets control to go inside

After P_j comes out then P_0 gets control to go back inside.

Two cases arise:

Case 1: P_1 doesn't want to enter CS, but because it's P_1 's turn, P_0 is stuck waiting. P_1 is forced to go in just to hand the turn back.

Case 2: CS is completely empty, P_0 is ready and waiting, but it still can't enter because $turn = 1$. An empty CS should mean any interested process can go in, but here it cannot.

Requirement	Result	Reason
Mutual exclusion	✓	Only the process whose turn it is may enter.
Progress	✗	The decision of which process enters is decided by turn, not by which process is interested.

Bounded waiting	✓	Each process waits at most one full cycle.
-----------------	---	--

Algorithm 2 : Flag array

The two processes use a boolean array, flag. A process sets flag[i] = true to declare intent, then waits while the other's flag is true.

```
flag[i] = true;
while (flag[j]);    // wait if other process wants CS
    /* critical section */
flag[i] = false; //i has now entered and exited, so it's false now
```

Requirement	Result	Reason
Mutual exclusion	✓	A process only enters when the other's flag is false.
Progress	✗	If both set their flags simultaneously, both spin forever : deadlock.
Bounded waiting	✗	The deadlock scenario means a process may wait indefinitely.

Algorithm 3 : Peterson's solution

Combines both flag[] and turn. A process sets its flag *and* sets turn to the other process, giving the other process priority. It waits only if *both* the other flag is true and it's the other's turn.

```
flag[i] = true; //declare intent
turn = j;
while (flag[j] && turn == j);    // wait only if other wants CS AND it's their turn
    /* critical section */
flag[i] = false;
```

Key insight: Setting turn = j means "if there's a conflict, let the other process go first." Only one process can "lose" the race to set turn, so deadlock is impossible.

Requirement	Result	Reason
Mutual exclusion	✓	At most one process passes the while condition.
Progress	✓	If only one wants the CS, it enters immediately. If both want it, turn decides.
Bounded waiting	✓	The waiting process enters as soon as the other exits and resets its flag.

These software solutions do not work on modern computer architectures. Modern CPUs don't always execute instructions in the exact order you wrote them, they reorder them for speed. Also, each CPU core has its own cache, so one core's write to flag or turn might not be immediately visible to another core.

Peterson's solution depends on both processes seeing each other's updates instantly and in order. Modern hardware doesn't guarantee this. Memory barriers are special instructions that force the CPU to sync up before moving on. Modern systems: hardware synchronization.

Hardware synchronization

Hardware solutions prevent race conditions by ensuring that critical sections are locked. They use atomic instructions to set a lock on the CS. **An atomic instruction is an uninterruptible instruction that happens all at once without any scope of interleaving.** A process acquires lock before entering the CS and releases the lock after leaving the CS

TestAndSet()

Atomically reads a boolean target (our lock) and sets it to true, returning the old value. A shared lock variable (initially false) is used. target=false, lock free. target=true, lock busy.

```
boolean test_and_set (boolean *target){
    boolean rv = *target;
    *target = TRUE;
    return rv;
}
```

If the lock was false, TestAndSet returns false and we enter. If another process already holds the lock, TestAndSet returns true and we spin. Bounded waiting is not guaranteed: a process could spin forever if others keep acquiring the lock.

```
do {
    while (TestAndSet(&lock))
        ; // do nothing

    // critical section

    lock = FALSE;

    // remainder section
} while (TRUE);
```

Test and set purpose: it checks if a process already locked the cs. If yes, we wait. If not, we set status to locked and we enter.

1. Mutual exclusion
 - Implemented
 - One process cannot enter unless the other unlocks
2. Progress: if CS is free, the spinning process catches lock = false immediately and enters.
3. Bounded wait: If multiple processes are waiting, there is no order in which they might be executed and one might end up waiting forever.
Unsatisfied.

Swap()

Atomically swaps two memory words. A local variable, key = true and a global variable, lock = false are used:

Every process starts with key = true (wanting to enter). If the cs is free, lock is false. We swap the values. If cs is occupied, the lock is true. We do nothing. When a process exits, lock is simply turned to false.

```
key = true;
while (key == true)
    Swap(&lock, &key); // atomic swap. If cs was occupied, swap would be
                        // meaningless. Keep spinning.
```

```
/* critical section */
```

```
lock = false;
```

Swap also does not satisfy the bounded waiting requirement.

Modified Bounded Waiting Implementation

A modified TestAndSet that adds a waiting[] array to fix the bounded waiting problem.

1. When waiting[i] is true, p_i says that it intends to enter
2. Key = true, CS is locked
3. Loop runs while waiting[i] and key: process is waiting and CS is locked
4. Key = testandset(&lock) keeps checking. As soon as lock becomes free, key becomes false and loop breaks.
5. waiting [i] becomes false as p_i is not waiting, it is in the CS now.
6. While exiting, the process scans from i+1 all processes.
7. When it finds the next waiting process, it sends it to CS and sets its wait[j] to false. In that case, lock is intentionally NOT set to false. Because ownership is directly handed over to another waiting process. This is called passing the baton

8. If no one is waiting, it comes back to itself and sets lock = false.

Now since we cyclically visit every waiting process, none waits indefinitely.

```
do {
    waiting[i] = TRUE;
    key = TRUE;
    while (waiting[i] && key)
        key = TestAndSet(&lock);
    waiting[i] = FALSE;

    // critical section

    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;

    if (j == i)
        lock = FALSE;
    else
        waiting[j] = FALSE;

    // remainder section
} while (TRUE);
```

Both above described hardware based solutions are complicated to apply in the real world. The solution is a semaphore.

Semaphores

A semaphore is an integer variable accessible only through two atomic operations:

- **wait(S)**
S represents the number of available resources/slots.
If $S \leq 0$, block the process and stop working.
Otherwise decrement S and proceed. Represents "acquiring" a resource.

```
wait(S) {
while S <= 0;
```

```
S--;  
}
```

- **signal(S):** Increments S. Indicates the finishing of an earlier started process. If processes are waiting, wake one. Represents "releasing" a resource.

```
signal(s){  
    S++;  
}
```

Types

Type	Range of S	Use
Counting semaphore	Any integer ≥ 0	Managing a pool of finite resources (e.g. N printers). Initialized to the number of available instances. When the count for the semaphore goes to 0, all resources are being used. After that, processes that wish to use a resource will block until the count becomes greater than 0.
Binary semaphore (mutex lock)	0 or 1	Mutual exclusion. Processes share a semaphore, so it is initialized to 1; one process holds it at a time.

Imagine a bathroom with multiple stalls. If we have 3 stalls, we want to ensure that up to 3 people can use them at once. We use a counting semaphore set to 3. Each time someone enters, the count goes down. When they leave, the count goes up. If all stalls are full (count is zero), the next person waits. The counting semaphore tracks multiple resources. Binary semaphore keeps track of each individual stall. It can either be full or empty. Ensures that only one person can use a stall at a time.

Using a semaphore for ordering (synchronization)

To guarantee that statement S2 in P2 always executes after S1 in P1. Let p1 and p2 share a common semaphore variable called synch.

```
semaphore synch = 0;
```

P1:

```
S1;  
signal(synch); // signal -> announce S1 completed
```

```
P2:  
wait(synch); // wait -> wait until S1 completes  
S2;
```

If P2 reaches wait first, it's blocked until P1 calls a signal. This pattern is useful for ordering any two events across processes.

Busy waiting vs. blocking

- **Busy waiting (spinlock):** the process continuously loops checking the condition. Wastes CPU cycles but avoids context-switch overhead, making it preferable when the expected wait is very short (e.g. kernel locks on a multi-core machine).

All examples we have seen so far in semaphore waiting are spinlocks.

- **Blocking semaphore:** instead of spinning, the process calls `block()` when a process is blocked (hit by `wait()`). `block()` moves it to a waiting queue and suspends it. When another process calls `signal()`, another function `wakeup()` moves a blocked process back to the ready queue. A negative semaphore value indicates how many processes are currently blocked.

A FIFO waiting queue ensures bounded waiting. The process that has waited longest is always woken next.

Deadlock & starvation

- **Deadlock:** Two or more processes wait forever, each holding a resource the other needs. Creates a circular dependency that no process can break on its own.
- **Starvation:** A process is indefinitely postponed because other processes keep being chosen ahead of it. Unlike deadlock, the system is still making progress; just not for the starving process.

Classical synchronization problems

Bounded buffer problem

Semaphores used are: `mutex = 1`, `full = 0`, and `empty = n`. `Mutex` protects the buffer from simultaneous access, `full` counts filled slots so the consumer waits if no item exists, and `empty` counts free slots so the producer waits if the buffer is full. `full` and `empty` are counting semaphores.

Producer process:

```
do {  
    /* produce an item in next_produced */  
  
    wait(empty);  
  
    wait(mutex);  
  
    /* add next produced to the buffer */  
  
    signal(mutex);  
  
    signal(full);  
} while (true);
```

Readers-writers problem

Multiple readers may read simultaneously. A writer needs exclusive access, no readers or other writers may proceed while it writes.

Access pattern	Safe?
Reader + Reader	Safe
Reader + Writer	Unsafe
Writer + Writer	Unsafe

Shared variables: `rw_mutex` (writer's lock), `mutex` (protects `read_count`), `read_count` (number of active readers). The first reader locks `rw_mutex`; the last reader unlocks it, allowing a waiting writer in.

1. First RW problem: Readers get priority

No reader will be kept waiting unless a writer has already obtained permission to use the shared object.

No reader should wait for other readers to finish simply because a writer is waiting.

2. Second RW problem: Writers get priority

Once a writer is ready, that writer performs its write as soon as possible.

If a writer is waiting to access the object, no new readers may start reading.

Data set, integer `read_count`, and semaphores `rw_mutex` and `mutex` are shared.

- **Rw mutex**
 - Initialised to 1
 - Common to both readers and writers
 - Mutual exclusion semaphore for the writers. When a writer enters, nobody else can access the CS.
 - Used by the first and last readers to enter/exit CS. When the first reader is in, the writers cannot enter. Only when the last reader exits, the writer can enter.
 - The semaphore is not altered by readers who enter or exit while other readers are in CS.
- **Mutex**
 - Initialized to 1
 - Ensures that `mutex` is maintained while `readcount` is updated.
- **Read_count**
 - Initialized to 0
 - Keeps track of number of readers inside CS.

Writer:

```
wait(rw_mutex)
/* write */
signal(rw_mutex)
```

Reader:

```

wait(mutex)
    read_count++
    if (read_count == 1) wait(rw_mutex)
signal(mutex)
/* read */
wait(mutex)
    read_count--
    if (read_count == 0) signal(rw_mutex)
signal(mutex)

```

1. In the first RW problem, the writer might starve
2. In the second WR problem, the reader might starve
3. Therefore, does not comply with bounded waiting

Dining philosophers problem

Five philosophers sit at a circular table with one chopstick between each pair. A philosopher needs both adjacent chopsticks to eat.

```
Semaphore chopstick[5] = {1,1,1,1,1};
```

Philosopher i:

```

wait(chopstick[i])          // pick up left
wait(chopstick[(i+1)%5])    // pick up right
/* eat */
signal(chopstick[i])
signal(chopstick[(i+1)%5])

```

If all five philosophers pick up their left chopstick simultaneously, all wait forever for their right chopstick; deadlock. Solutions:

- Allow at most 4 philosophers to sit at a time. This ensures at least one philosopher can always obtain both chopsticks, preventing deadlock.
- Asymmetric pickup: odd-numbered philosophers pick left then right; even-numbered pick right then left.
- Pick up both chopsticks only if both are available (atomic check).

Semaphore pitfalls

Semaphores are powerful but error-prone. Common mistakes and their consequences:

Mistake	Effect
---------	--------

signal(mutex) before CS, wait(mutex) after	Mutual exclusion violated : multiple processes enter the CS simultaneously
wait(mutex) ... wait(mutex) (double wait)	Deadlock : process blocks itself permanently
Missing signal(mutex)	Deadlock for all processes waiting on mutex
Missing wait(mutex)	Mutual exclusion violated

Q1)The program follows to use a shared binary semaphore T.

Process A

int Y;

A1: Y = X*2;

A2: X = Y;

signal(T);

Process B

int Z;

B1: wait(T);

B2: Z = X+1;

X = Z;

T is set to 0 before either process begins execution and, as before, X is set to 5.

Now, how many different values of X are possible after both processes finish executing?

- a) one
- b) two
- c) three
- d) four

ans=1

Monitors

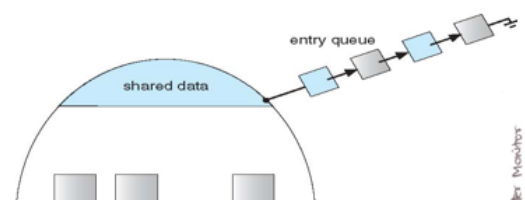
A monitor is a high-level language construct that bundles shared variables, procedures, and initialization code into a single module. The compiler enforces mutual exclusion automatically; only one process can be executing inside the monitor at any time. This eliminates the manual wait/signal bookkeeping that makes semaphores error-prone.

```
monitor monitor-name{

    // shared variable declarations

    procedure P1 (...) { .... }
    procedure Pn (...) {.....}
    Initialization code (...) { ... }
}
```

Internal variables are accessible only by code within the procedure. Procedure defined within the monitor can only access the variables that are locally declared within the monitor.



The entry queue represents processes lined up. The circle represents a monitor. While they wait in the entry queue, only one process is able to enter the monitor. This ensures mutual exclusion.

Analogy: security checkup at airports

The process outside cannot see variables inside the monitor but can call procedures (to check if they can enter) of the monitor. In a monitor, we don't need to manually write code for locking/unlocking.

It is not powerful enough to model synchronization schemes. Like multiple readers and one writer etc. are not supported. To get this fixed, we have condition variables.

Condition variables

Declared as condition x , y inside a monitor. They support only two operations:

- $x.wait()$: This function suspends the calling process and releases the monitor, allowing another process to enter.
- $x.signal()$: resumes exactly one process waiting on x . If none are waiting, it has no effect (unlike semaphore signal, which increments a counter).

When a process calls $x.wait()$, it steps out of the room and sits in the x waiting area. When another process calls $x.signal()$, it wakes exactly one process from that waiting area and lets it re-enter.

If process P invokes $x.signal()$, and process Q is suspended in $x.wait()$, we cannot run them together. If a process signals, the waiting process wants to run, but so does the signaller. So, "signal and wait" means the signaller steps aside, letting the waiting process run immediately. "Signal and continue" means the signaller keeps going, and the waiting process will get its turn later.

- Signal-and-wait: Signaling process suspends until the resumed process exits or waits again.
- Signal-and-continue: Signaling process continues; resumed process waits until signaller exits the monitor.

Advantages of Monitor

1. Makes parallel programming easy

2. Less error prone

Disadvantages of Monitor

1. Have to be implemented as part of the programming language
2. Compiler must generate code
3. The compiler has to pick and implement the right system-level tools to manage synchronization properly.

Monitor solution to Dining Philosophers Problem

monitor DiningPhilosophers

```
{
    enum { THINKING; HUNGRY, EATING) state [5] ;
    condition self [5];

    void pickup (int i) {
        state[i] = HUNGRY;
        test(i); //check if left and right are not eating. If true,
state[i] = EATING
        if (state[i] != EATING)
            self[i].wait;
    }
}
```

Execution of Pickup(), Putdown() and test() are all mutually exclusive, i.e. only one at a time can be executed. Deadlock never occurs but starvation is possible.

putdown-> sets status back to thinking and releases both chopsticks. If either neighbour was waiting, get them working.

Semaphore vs. monitor

	Semaphore	Monitor
Level	Low-level	High-level construct
Mutual exclusion	Manual (wait/signal)	Automatic (compiler-enforced)
Error risk	High : ordering mistakes cause bugs	Low : structure prevents most errors

Language support	Not required	Required (Java, C#, etc.)
Condition signaling	Increments counter; buffered	Wakes one waiter; no buffering

Module 3.3

Deadlocks

System Model

- Finite resources, many processes. These have to be distributed among competing processes.
- Types: CPU, memory, files, I/O
- Resources are categorized per type, each type has multiple instances
- A process may utilize a resource in only the following sequence:
Process flow: request → use → release

Request

1. Process requests the resource
2. If the resource cannot be granted (if it's being used already)
3. The requesting process waits

Use

1. Process operates on resource

Release

1. After done, the process releases resource

Deadlock

- All processes wait forever
- Caused by circular waiting
- Can involve same or different resources

4 Conditions

- Mutual exclusion → one process at a time uses resource
- Hold and wait → process holds one resource, waits for others
- No preemption → process cannot force another process to release
- Circular wait → cycle of waiting, p1 waits on p2 waits on p3 waits on p1

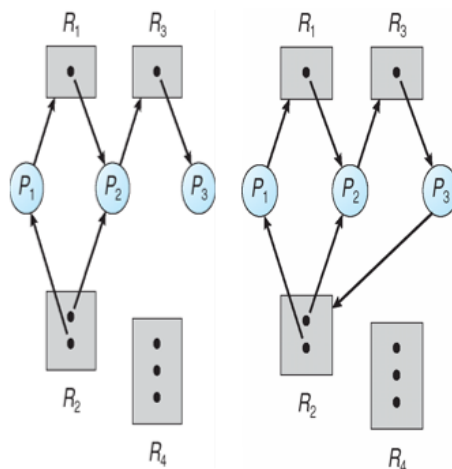
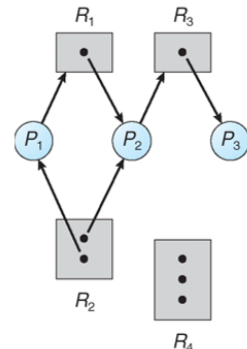
Deadlock can arise if four conditions hold simultaneously.

RESOURCE ALLOCATION GRAPH

- Nodes are processes and resources
- $P \rightarrow R$ means request
- $R \rightarrow P$ means allocated
- No cycle means no deadlock
- Cycle means possible deadlock
- If one instance per resource, cycle means deadlock

P1 holds an instance of R2 and is waiting on R1. P2 is holding an instance of R1 and is waiting on R3. P3 is holding on R3.

Since there are no cycles, there is no deadlock.



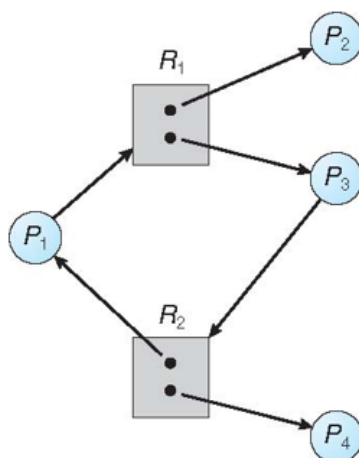
In this graph, we notice two cycles.

$r2 \rightarrow p1 \rightarrow r1 \rightarrow p2 \rightarrow r3 \rightarrow p3 \rightarrow r2$ is one cycle

And $r2 \rightarrow p2 \rightarrow r3 \rightarrow p3$ is the second cycle.

P1, p2, p3 are deadlocked by circular wait.

Exception cases:



In this example, even though there is a cycle: $P1 \rightarrow R1 \rightarrow P3 \rightarrow R2 \rightarrow P1$, there is no deadlock.

Why?

Because P4 needs r2 alone. It takes r2, completes the process, and frees it. Now, we have two instances of R2 that can be used by p1 and p3 in sharing. Same for P2 with R1.

METHODS TO HANDLE

1. Deadlock Prevention
2. Deadlock Avoidance
3. Detection and recovery
4. Ignore:pretend that deadlocks never occur (used in UNIX)

DEADLOCK PREVENTION

Break any one condition by constraining how requests for resources can be made.

Mutual Exclusion

Allow processes to share resources to break mutual exclusion

Hold and Wait

- Before the process even begins running, it declares everything it will ever need. The OS either gives it all at once or makes it wait until everything is available. This is impractical.

A process that runs for an hour might only need the printer in the last 5 minutes. But it's holding the printer the entire hour. Nobody else can use it. And in reality, a process rarely knows everything it'll need before it starts.

- A process can ask for a resource, but only if it currently has zero resources allocated to it. If it needs something new, it must release everything it has first, then request the new set. So you can never be in a state where you're holding A and waiting for B. You'd have to drop A first.

Disadvantages

- a. Resource utilization is slow
- b. Starvation: A process that needs several popular resources might wait forever because every time it tries to grab them all together, some are taken.

No Preemption

Preempt resources to break no preemption. If a process is holding resources and it cannot begin working because it needs more resources, let all of its

holding resources be free. The process is only restarted when it can regain all its old resources.

Ways:

1. Simply free all resources being held by a waiting process till all its needed resources are available.
2. Or when a process is waiting on resources, check if it's held by another waiting process. If yes, free those resources and allocate to this process.

Circular wait

1. Order resources to break circular wait
2. Disadvantages are low utilization and starvation
3. Requires that each process requests resources in an increasing order of enumeration
4. A process can initially request any number of instances of a resource type, R_i . After that, the process can request instances of resource type R_j if and only if $F(R_j) > F(R_i)$

DEADLOCK AVOIDANCE

- Side effects: low device utilization and low system throughput.
- System stays in safe state
- Needs prior knowledge of max resources
- Resource-allocation state is defined by the number of available resources, the number of allocated resources, and the maximum demands of the processes
- Safe state means all processes can finish without a deadlock
- Every time a new request for a resource is released, the system must check if this resource's allocation will make the system unsafe.
- Unsafe state means risk of deadlock

When we only have single instances of resources, we use a resource allocation graph to check for deadlock avoidance. If we have multiple instances of resources, we use the Banker's algorithm.

RESOURCE ALLOCATION GRAPH

1. Let a claim edge represent that a process MAY request the resource.
2. Claim edge converts to request edge when a process actually requests the resource.

3. Request edge is converted to assignment edge if the resource gets allocated
4. When the resource is released, the edge reconverts to claim edge.
5. Resources must be claimed a priori. This implies that it declares resources before using them.

BANKER'S ALGORITHM

1. Used when multiple instances exist
2. Available vector holds free resources
3. Max is maximum demand that will ever be made
4. Allocation is current allocation
5. Need matrix = Max - Allocation
6. When a process gets all its resources, it must return in a finite time.
7. Checks for safety, simulates the allocation of a predetermined max number of resources.

Code with documentation:

<https://github.com/ashwera/SYNotes/blob/master/Semester%202/Operating%20Systems/OS%20ST/bankersAlgorithm.cpp>

Practice questions

Process	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	2	1	5	2	0
P1	1	0	0	0	1	7	5	0				
P2	1	3	5	4	2	3	5	6				
P3	0	6	3	2	0	6	5	2				
P4	0	0	1	4	0	6	5	6				

Consider the following snapshot of a system-

Answer the following questions using the Banker's algorithm-

- What are the total instances of Resources
- What is the content of the matrix need?
- Is the system in a safe state?
- If a request from process P1 arrives for (0,4,2,0), can the request be granted immediately?

DETECTION

- System allows deadlock to occur
- Single instance resource type uses wait-for graph with the resource allocation graph. Maintain wait-for graph Nodes are processes P_i P_j if P_i is waiting for P_j
- Cycle means deadlock
- Multiple instances resource type use matrix method

Detection Algorithm

<https://github.com/ashwera/SYNotes/blob/master/Semester%202/Operating%20Systems/OS%20ST/detectionAlgorithm.cpp>

- Start with available resources (work)
- Mark all processes unfinished
- Find process where request $\leq$ work
- If found, "run" it
- Add its allocation resources back to work vector
- Mark the process finished

7. Repeat steps 3-6
8. If no process fits → stop
9. Unfinished processes = deadlock

Difference between Detection and Banker

Purpose	Banker's Algorithm	Detection Algorithm
Goal	Avoid deadlock	Detect deadlock
When used	Before allocation	After allocation
Input needed	Max, Alloc, Available	Alloc, Request, Available
Core check	$\text{Need} \leq \text{Available}$	$\text{Request} \leq \text{Available}$
State	Checks safe state	Finds stuck processes
Action	May deny request	Just reports deadlock
Outcome	Prevents deadlock	Identifies deadlock
Idea	"Stay safe"	"Who is stuck"

RECOVERY

- Terminate processes

2 methods

1. Abort all deadlocked process
 - a. Breaks the cycle but at a high cost
 - b. Some processes may have already computed for some time
2. Abort one by one till a deadlock cycle is eliminated
 - a. High overhead
 - b. Deadlock detection algorithm needs to be invoked

Aborting a process might be a questionable choice. It might leave files in a corrupted state if the file was in update state while the process got terminated. If the process was in midst of printing data, it might reset the printer. So we abort the process which will incur the least cost.

What order?

1. Priority of process

2. Time that process has already spent
3. Resources it has used
4. Resources it needs
5. How many processes need to stop for this one
6. Is this process interactive/batch

Other option:

Preempt resources from one/more deadlocked processes.

Give this resource to some other process till the deadlock is broken.

1. Selecting a victim
 - a. Check what resource and what process is preempted
 - b. Determine order of preemption
 - c. Cost factors: number of resources held by a process, amount of time so far consumed
2. Rollback
 - a. Return to some already met safe state, restart from that state
 - b. Process runs ; OS keeps checkpoints.
 - c. Error / deadlock happens ; undo changes.
 - d. Memory restored to earlier safe state.
 - e. If it is hard to determine what a safe state is, the simplest solution is complete rollback. Abort and restart the process.
3. Starvation
 - a. Same process can be victimized again and again
 - b. To guarantee that this does not happen, ensure that the same process is picked as victim a finite number of times only

Interprocess Communication

A process within a system may be independent or cooperating.

- Independent: cannot affect nor be affected by another process. It does not share any data with another process
- Cooperating: can be affected or affect another process and usually shares data with another process.

What gives birth to a cooperating process?

Information Sharing

Several users might need the same information

We provide an environment to enable concurrent access to such information

Computation speedup

We break a task into smaller subtasks to make its computation faster. Each of these execute parallelly with others. A computation speedup can only be achieved if a computer has multiple processing elements.

Modularity

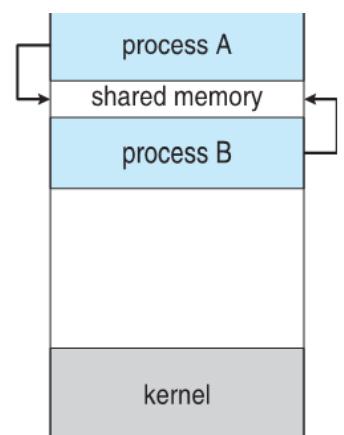
The system is constructed modularly. In this method, we divide the system functions into separate processes/threads.

Cooperating processes need an IPC (interprocess communication) mechanism to exchange data and information.

Models of IPC

Shared Memory

One region of memory shared between the cooperating processes is established. Processes can exchange information by working on the shared region. This method allows maximum speed and convenience. System calls are only required once to establish a shared memory region. Once established, all accesses are direct and no need to take any assistance from the kernel.



Message passing

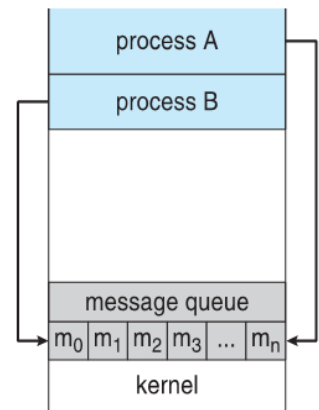
Communication occurs by means of messages exchanged between the two processes.

It exchanges smaller amounts of data to avoid conflict. It is easier to implement than shared memory.

It is slower because messages are implemented by system calls. System calls need more kernel intervention = less speed

Why do we need?

Communicating processes live on different computers connected by a network



The IPC provides two operations. `send(msg)` and `receive(msg)`. Msg size can be either fixed or variable. For fixed size msg, system implementation is easy but programming is hard. The OS knows exactly how much memory to allocate for every message buffer – no dynamic allocation needed. But If your data is smaller than the fixed size → you waste space and must pad the message. If data is large, you need to split across different data structures. For variable, system implementation is hard and programming is easy.

For P and Q to communicate, we need

1. A communication link
2. Exchange msgs via send and receive functions

How is a communication link implemented?

1. It can be physical
 - a. **Shared memory** – both processes read/write the same region of RAM
 - b. **Hardware bus** – data travels over the system bus connecting components
 - c. **Network** – data travels over a network connection, could be same machine or different machines
2. It can be logical
 - a. **Direct or indirect** – do processes name each other explicitly, or do they communicate through a mailbox/queue in the middle
 - b. **Synchronous or asynchronous** – does the sender wait for the receiver to receive before moving on, or does it send and continue immediately
 - c. **Automatic or explicit buffering** – does the system automatically hold messages in a queue, or does the programmer manage that

Physical = what the link is made of

Logical = how the link behaves

Direct vs Indirect

- Direct → processes name each other explicitly.
 - `send(P,msg)` -> send msg to P.
 - Only need to know identity to communicate
 - Link is associated with one pair of communicating process.
 - Only one link between two processes.
 - Both sender and receiver name the other process to communicate.
 - A variant: the sender mentions which process receives its message but receiver does not.
- Indirect → messages go through a shared mailbox/port
 - Messages are directed towards and received from mailboxes.
 - Each mailbox = unique id.
 - Processes can only communicate if they have a shared mailbox.
 - Links are only established if the two processes have a common mailbox
 - Link might be unidirectional or bidirectional.

Synchronous vs Asynchronous

- Synchronous → sender/receiver **blocks** until the other is ready
 - **Blocking send:** sending process is blocked until msg is received by either the receiver or the mailbox
 - **Non-blocking send:** sending process sends msg freely
 - **Blocking receive:** receiver is blocked unless a message can be received
 - **Non-blocking receive:** the receiver either returns a valid or a null value.
 - Both send and receive are blocked-> keep spinning between S and R
 - Solution to producer-consumer becomes trivial
 - Producer invokes the blocking `send()` call and waits until the message is delivered to either the receiver or the mailbox. Consumer invokes `receive()`, it blocks until a message is available.
- Asynchronous → sender/receiver **continues** without waiting

Automatic vs Explicit Buffering

- Automatic → OS provides unbounded queue, sender never blocks
- Explicit → fixed/zero capacity queue, sender may block if full

Module 4.1

File Management

File & File System

- File = collection of data created by users
- File System = OS component that manages files

Why files matter:

- Store data permanently
- Share data between processes
- Organize data properly

Properties: Long-term existence, sharable between processes, structured/hierarchical

File Operations

- Create
- Delete
- Open
- Close
- Read
- Write

File Structure Terms

A file can be structured as a collection of records or as a sequence of bytes. UNIX, Linux, Windows, Mac OS's consider files as a sequence of bytes. Other OS's adopt the collection-of-records approach. This approach is useful for DBs.

- Field
 - smallest data item (Name, DOB, Salary)
 - has datatype
 - fixed/variable length
- Record
 - collection of related fields (e.g. Employee: Name, ID, Salary, Dept)
 - One field is the key field

- Key Field has a unique identifier (e.g. Employee ID)
- Can be either fixed length or variable
- File
 - collection of similar records
 - treated as single unit with access control
 - Can be referenced by name
 - Access control is applied at this level
- Database
 - Collection of related files with explicit relationships (e.g. College DB: Students, Faculty, Marks)

File Management System (FMS)

System software providing file services is called an FMS. Users access files through FMS only.

Objectives:

1. Meet user requirements
2. Guarantee validity of data in files
3. Optimize performance
4. Provide I/O support for a variety of storage device types
5. Prevent data loss
6. Provide standard I/O
7. Provide I/O support for multiple users in the case of multiple-user systems

File System Architecture

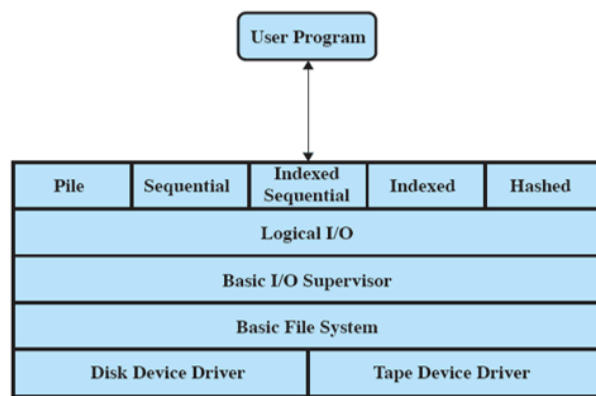


Figure 12.1 File System Software Architecture

Layer	Role
Access Method (pile/sequential/etc.)	Interface between app and file system
Logical I/O	Works with records
Basic I/O Supervisor	Controls I/O, scheduling, buffering
Basic File System	Handles raw blocks only
Device Drivers	Communicates with hardware

Device Drivers

- It is at the lowest level
- Communicates with hardware
- Responsible for starting IO operations
- Processes the completion of an IO request
- Goal: Initiate I/O operations and process their completion by communicating directly with hardware
- Is a part of the OS

Basic file System

- Also called Physical I/O Level.
- Acts as the primary interface between computer systems and external storage devices.
- Deals with blocks of data exchanged with disks or mass storage devices.
- Functions:
 - Placement of blocks on secondary storage.
 - Buffering blocks in main memory.
- Considered part of the Operating System.
- Its goal is to handle and exchange blocks of data between secondary storage and memory buffers

Basic IO supervisor

- Responsible for all IO initiation and termination
- It maintains all control structures related to: Device I/O, Scheduling, File status (which files are open, their current state)
- Selects device on which IO is to be performed
- Schedules disk and tape accesses to optimize performance
- Assigns I/O buffers and allocates secondary memory

- Its core goal is optimizing I/O performance.
- Also part of the OS

Logical IO

- Enables users/applications to access files
- Provides a general purpose record IO. It means Logical I/O gives applications a simple way to read/write records without worrying about how data is stored in blocks underneath.
- Maintains basic data ABOUT file. Metadata.
- It is the interface between the logical commands sent by a program and the physical details required by disk.

Access Method

- This level of file system is closest to the user
- It provides an interface between apps and the file systems/devices holding data
- Different access methods reflect different file structures and different ways of accessing and processing the data

File Organization

File organization is the logical arrangement of records based on the access method.

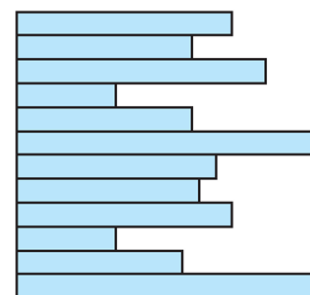
Criteria for choosing file organization:

1. Short access time.
2. Easy updates.
3. Storage economy.
4. Simple maintenance.
5. Reliability.

Note: These criteria may conflict; for example, indexes improve speed but use extra storage. The priority of criteria depends on application that uses the file.

1. Pile File

- Data stored in arrival order, no structure
- Searching: exhaustive linear search
- Easy updates, good storage use
- Very slow searching

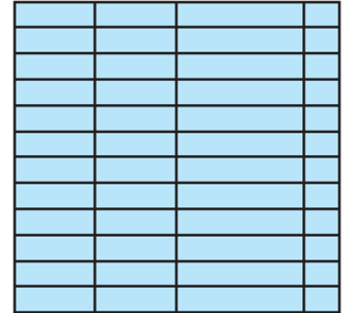


Variable-length records
Variable set of fields
Chronological order

- Used when data is collected before processing
- Each record = one burst of data

2. Sequential File

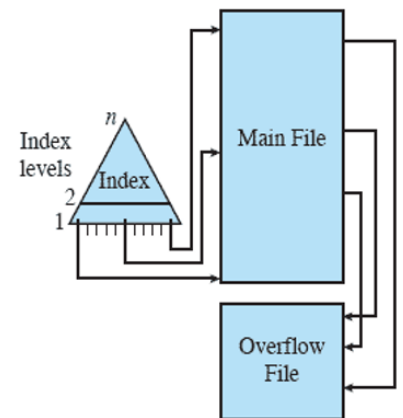
- One field (usually the first field in every record) is called the key field
- Key field uniquely identifies each record
- Records are stored in key sequence. The keys are ordered alphabetically or numerically.
- Good for batch processing (payroll, billing)
- Stored on tape and disk
- Slow random access, difficult insertions since the new key might be in between two already existing records. To find that and insert in that sequence is an overhead.



Fixed-length records
Fixed set of fields in fixed order
Sequential order based on key field

3. Indexed Sequential File

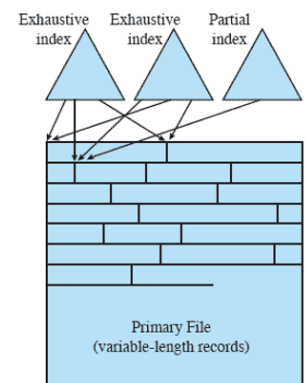
- Records are organized in sequence based on key
- Adds an index to support random access
- Adds an overflow file. When a new record is inserted, it goes into the overflow file (instead of disturbing the main sequential file). Later, this is merged during reorganization. Faster insertion.
- Each record = key + pointer → dramatically reduces searches
- Example: 1M records → 500,000 avg searches → ~1,000 with index
- Multiple index levels reduce searches further (500,000 → 1,000 → 150)



(c) Indexed Sequential File

4. Indexed File

- Multiple indexes for searching by many attributes (ID, Name, Dept)
- No restriction on where a record is as long as a pointer points to it
- Variable-length records allowed
- Index types:
 - Exhaustive (every record has an index)
 - Partial (selected records have an index)
- Very fast, flexible access



(d) Indexed File

- All indexes must be updated
- Used in: Airline reservations, inventory systems

5. Direct / Hashed File

- Employ a hash function on key, function gives direct storage location
- No sequential ordering, fixed-length records preferred
- Fastest direct access
- Used in: Directories, pricing tables, schedules. Used when very fast access is required and fixed length records are used.

Quick Comparisons

Type	Strength	Weakness
Sequential	Simple, good for batch	Slow random access
Indexed Sequential	Faster via index	More complex
Indexed File	Multi-attribute search	Index overhead
Hashed	Fastest direct access	No ordering

File Directory

A directory: Stores file metadata like Name, Type, Location, Size, Ownership, Permissions. A file directory can be seen as a folder that holds many files.

Operations Performed on a Directory

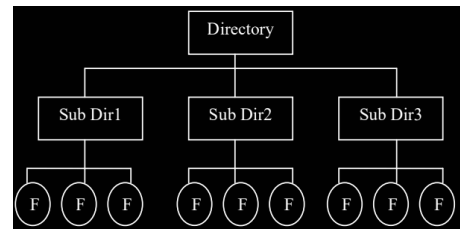
Understanding directory operations helps define the requirements for a file structure. Key operations include:

- Search – find a file entry in the directory
- Create file – add a new entry to the directory
- Delete file – remove an entry from the directory
- List directory – display contents and file information
- Update directory – modify file attributes

Directory Structures

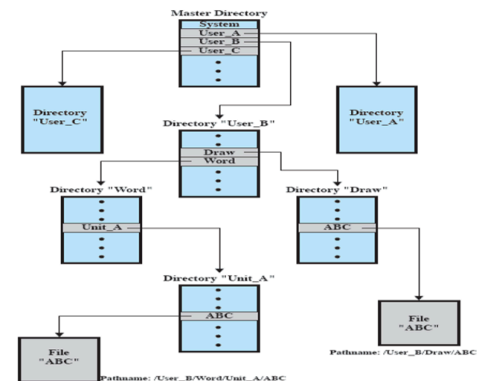
Two-Level Scheme

- A master directory sits at the top
- Each user gets their own directory underneath it
- File names only need be unique within a single user's directory
- File directory can control who accesses which folder
- Each user's subdirectory can have multiple directories also. But only one more level. Beyond that, we get tree-structured directory.



Tree-Structured Directory

- Master directory with user directories underneath
- Each user directory can have subdirectories and files as entries
- Subdirectory = a directory inside another directory



File Sharing & Access Rights

When files are shared, access must be controlled and managed across simultaneous users. Access rights assigned to users are as follows:

Right	What it allows	Example
None	User can't even see the file exists	System security files in PCs
Knowledge	User can see the file exists and who owns it, and can request more access	Shared company doc that you don't have access to
Execution	Can load and run a program, cannot copy it	GTA6
Reading	Can read, copy, and execute the file	Shared company doc that you only have read access to

Appending	Can add data, but cannot modify or delete existing content	Database with only insert rights
Updating	Can modify, delete, and add to file data	Applications on phone
Changing Protection	Can change access rights of other users	Company administrators
Deletion	Can delete the file from the system	Code developers

User access rights

Owner

- Has full rights
- May grant rights to others
- Usually creator of file

Specific users

- Are designated by userID
- Only have rights given to them

User groups

- Set of users not individually defined

All

- Users that have access to the system
- Public files

Secondary Storage Management

- A file on secondary storage = collection of blocks
- OS/FMS is responsible for allocating blocks to files. Decides where each piece of the file is stored on a disk.
- Two key issues:
 - Allocating space to files
 - Tracking space available for allocation

File Allocation Basics

- Disk divided into physical blocks (sectors on a track)
- Files divided into logical blocks
- Logical block size = some multiple of physical block size
- File Allocation Table (FAT) = data structure tracking portions assigned to each file

Preallocation vs Dynamic Allocation

Preallocation: maximum file size declared at creation time. Imagine you assume 10k admissions this year at somaiya. So you set aside room for 10k application forms. But what if you get 15k? What if you only get 2k?

- Problem: hard to estimate size accurately, tends to waste space

Dynamic Allocation: Space allocated in portions as needed. Allocate space as you go.

- More efficient, no upfront size commitment

Portion Size Trade-offs

Bigger chunks make the whole system run smoother while smaller chunks make the management of a single file better.

- Contiguous space/ few larger portions → better performance, especially for sequential reads
- Many small portions → larger management tables needed
- Fixed-size portions → easier reallocation
- Variable/small portions → less wasted storage

File Allocation Methods

1. Contiguous Allocation

- Single contiguous set of blocks allocated at file creation (preallocation)
- FAT (file allocation table) needs only one entry per file: starting block + length
- Best performance for sequential files, easy block retrieval
- External fragmentation over time. Difficult to find contiguous files of sufficient size.
- Solution: From time to time, it will be necessary to perform a compaction algorithm to free up additional space on the disk. A compaction

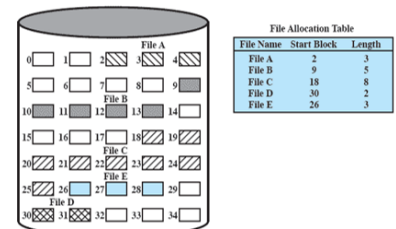


Figure 12.9 Contiguous File Allocation

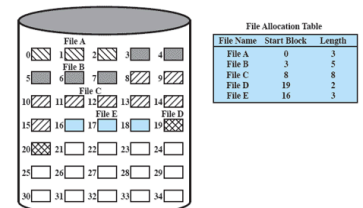


Figure 12.10 Contiguous File Allocation (After Compaction)

algorithm rearranges scattered memory or disk blocks so all free space becomes one continuous block.

2. Chained Allocation

- Allocation on individual block basis
- Each block has a pointer to the next block in the chain
- FAT needs only: starting block + length
- Any free block can be added to the chain – no external fragmentation
- Better for sequential files
- Drawback: Programs used closely should be stored closely. This is not implemented in chained allocation. To overcome this, the system occasionally reorders blocks to improve space or access efficiency. Pointer overhead is also a drawback.

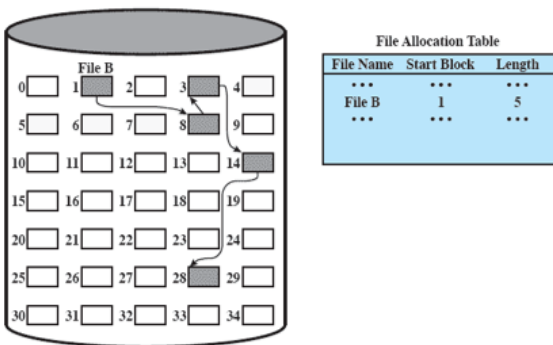


Figure 12.11 Chained Allocation

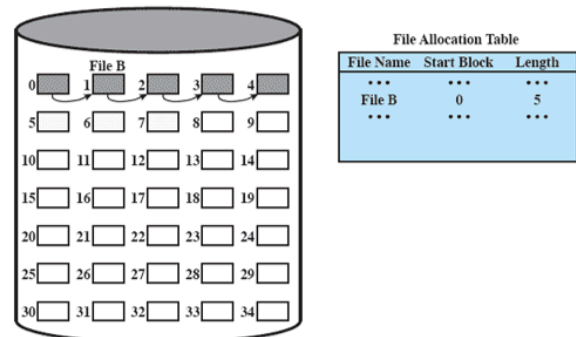


Figure 12.12 Chained Allocation (After Consolidation)

3. Indexed Allocation (Block Portions)

- FAT contains a separate one-level index for each file. The index is a single table that lists all the blocks that a file uses.
- Each index entry points to a disk block of the file. Tells where on the disk that part of the file lives.
- File index stored in a separate block; FAT entry points to that block.
- The system stores a small table for each file that maps out all its parts. When you want to access the file, the system looks up this index, finds where each piece is on the disk, and uses that to read the file in order.

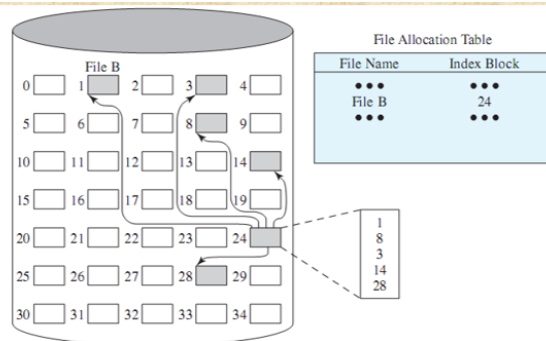


Figure 12.13 Indexed Allocation with Block Portions

4. Indexed Allocation (Variable Length Portions)

- Same concept but portions are variable in size

	Contiguous	Chained	Indexed	
Preallocation?	Necessary	Possible	Possible	
Fixed or variable size portions?	Variable	Fixed blocks	Fixed blocks	Variable
Portion size	Large	Small	Small	Medium
Allocation frequency	Once	Low to high	High	Low
Time to allocate	Medium	Long	Short	Medium
File allocation table size	One entry	One entry	Large	Medium

Free Space Management

Unallocated space must also be tracked. A disk allocation table is needed alongside the FAT for this purpose.

Methods:

Bit Tables (Bit Vectors)

- One bit = one disk block
- 0 = free, 1 = in use

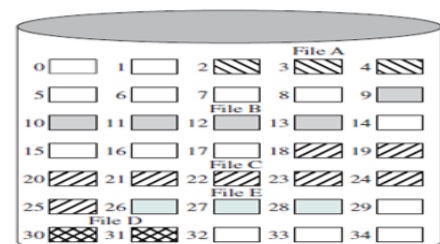


Figure 12.9 Contiguous File Allocation

- For the figure alongside, the bit vector would be
0011100001111100001111111111011000
- Advantages
 - Easier to find a contiguous group of free blocks
 - Works well with any file allocation method
 - As small as it gets. It needs very little memory compared to storing full addresses or pointers.

Chained Free Portions

- Free portions linked together via pointer + length value
- Negligible space overhead, no disk allocation table needed, just a pointer to start of chain
- Suited to all file allocation methods
- Variable-length allocation means free spaces can have different sizes. Use the First-fit algo: Pick the first free space big enough.
- Disadvantages: it leads to fragmentation.

Indexing

- Treats free space as a file, uses an index table
- Best done on variable-size portions for efficiency
- Supports all file allocation methods
- Suppose disk blocks are: 1 2 3 4 5 6 7 8 9 10
Used blocks: 1, 2, 5, 7
Free blocks: 3, 4, 6, 8, 9, 10
The free-space index table stores: 3 → 4 → 6 → 8 → 9 → 10
- The free-space index table contains a pointer for each free block.

Free Block List

- Each block has a number assigned sequentially
- The list of all numbers of all free blocks = stored in a reserved portion of disk
- This list is usually huge and cannot be stored in main memory. Thus it is kept on disk.
- Only a small portion kept in memory using:
 - Stack: When a new block is allocated, it is popped from the top of the stack, which is in main memory. At deallocation, block is pushed on stack.
Only need to transfer between disk and main memory when in-memory portion is either full or empty.

- Queue: Allocate from head of queue, deallocate to tail. Transfer when head portion is empty or tail portion is full

UNIX File Management

Six File Types in UNIX

Mnemonic Smart Students Study Really Difficult Problems

1. Regular files: Store normal user data and programs. Internally treated as a stream of bytes.
2. Directories: Store information about files and folders.
3. Symbolic links (soft links): A data file containing the name of the file it is linked to
4. Pipes: Allow communication between processes. One process writes, another reads.
5. Special files (block/character devices): Represent hardware devices as files.
6. Sockets: is a special file which allows for advanced inter-process communication.

Symbolic Links (Soft Links)

- References another file using its path
- Appears to have its own name, but read/write operations redirect to the target file
- Deleting the symlink → original file unaffected
- Deleting/moving the original → symlink breaks
- Shown as l in `ls -l` output

How to find out what file type it is?

The easiest way to find out file type in any operating system is by looking at its extension such as .txt, .sh, .py, etc. If the file doesn't have an extension then in Linux we can use file utility.

CLI: `file filename`. Tells the type of file.

Inodes

- All UNIX file types are managed via inodes
- Inode (index node) = data structure describing a file or directory
- Stores: file attributes + disk block locations
- An inode (index node) is a control structure that contains the key information needed by the operating system for a particular file

- On disk: inode table contains inodes of all files

FreeBSD Inode Contents

FreeBSD is a free and open-source Unix operating system. It includes:

- File type and access mode
- Owner and group IDs
- Timestamps: created, last read, last written, last inode update
- File size in bytes
- Sequence of block pointers: a list of pointers that point to the actual data blocks on disk where the file's content is stored.
- Number of physical blocks used (including indirect pointer blocks)
- Number of directory entries referencing the file
- Kernel and user-settable flags
- Generation number (randomly assigned each time inode is allocated to a new file)
- Block size of data blocks referenced by inode

UNIX File Allocation

- Block-based, dynamic allocation (not preallocation)
- Indexed method used: part of index stored in the inode. The inode doesn't store all the block pointers directly – for large files that would be too many pointers.
- Inode includes: several direct pointers + three indirect pointers (single, double, triple)

UNIX Directories & Inodes

- Directories structured as a hierarchical tree
- A directory = a file containing a list of filenames + pointers to their inodes
- Each directory entry contains:
 - File/subdirectory name
 - i-number (index number) → used to index into the inode table

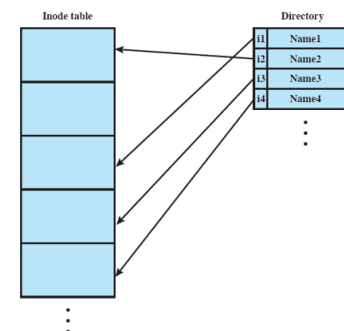


Figure 12.17 UNIX Directories and Inodes

Volume Structure

Layout of a UNIX file system on a disk partition is called its volume structure.

- Boot block: Contains code needed to boot the operating system.

- Superblock: Stores information and attributes of the file system.
- Inode table: Contains inodes storing metadata of each file.
- Data blocks: Store actual file and directory data.

UNIX File Access Control

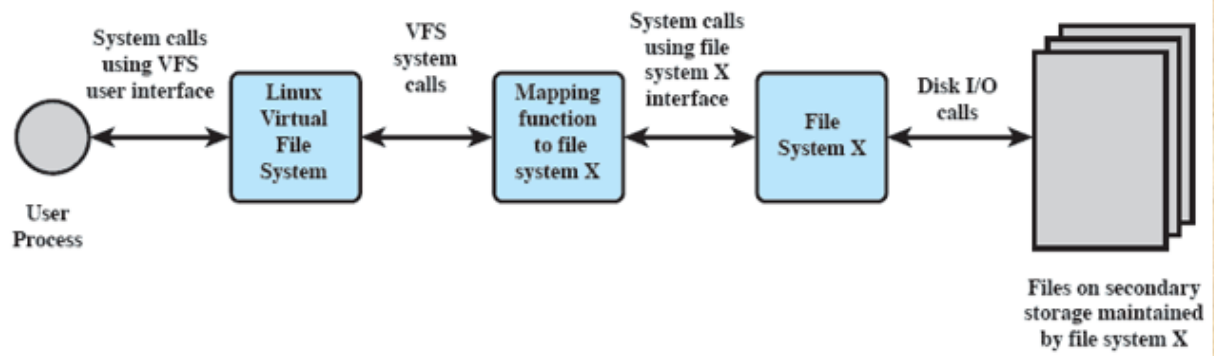
- Traditional UNIX: read/write/execute permissions for owner, group, others
- FreeBSD ACL (Access Control Lists):
 - Admin can assign lists of user IDs and groups to a file
 - Each user/group gets its own set of three protection bits (read, write, execute)
 - Files include an extra protection bit indicating whether an extended ACL exists

Linux Virtual File System (VFS)

A layer inside Linux that sits between user processes and the actual file system.

Its job: no matter what file system is underneath (ext4, NTFS, FAT32 etc.), the user process always sees the same simple interface.

- Versatile → supports many different file systems simultaneously
- Uniform interface → user processes interact with VFS the same way, always, regardless of what's below
- Common file model → VFS defines a standard template that any file system can be mapped to
- Basic properties assumed → every file, on any file system, on any hardware, is treated as having the same fundamental characteristics



Linux Virtual File System Concept

How VFS Works

1. User process issues a file system call (e.g., read) via VFS
2. VFS converts it into an internal kernel file system call
3. That call is passed to a mapping function for the specific file system (e.g., IBM's JFS)
4. Mapping function translates calls from one file system scheme to another

Gist: VFS acts as a translator – no matter what file system a file comes from, VFS converts it into a standard format the OS kernel always understands.

VFS Implementation

- Object-oriented scheme written in C
- VFS objects implemented as C data structures (since C doesn't natively support OOP)
- Primary objects in VFS: **FIDS**
 - **Superblock object:** Represents a mounted file system.
 - **Dentry object:** Represents a directory entry.
 - **Inode object:** Represents a specific file and its metadata.
 - **File object:** Represents an open file being used by a process.

Module 4.2.1

Disk Scheduling

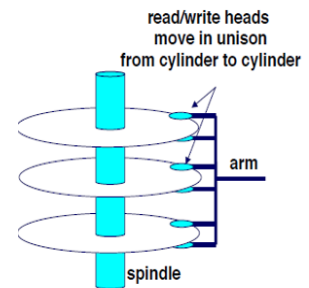
Hard Disk

An electro-mechanical device that stores and retrieves digital data using magnetic storage on one or more rigid, rapidly rotating platters coated with magnetic material.

Platters are paired with magnetic heads arranged on a moving actuator arm, which read and write data to the platter surfaces. Data is accessed in random-access manner -- individual blocks can be stored and retrieved in any order.

Key Terms

- **Disk arm:** The disk arm is the physical component that holds the read/write head and moves it across the platters to the correct track.
- **Seek Time** – time taken to move the disk arm to the required track (smaller seek time = faster access)
- **Rotational Latency** – time taken for the required sector to rotate under the read/write head
- **Transfer Time** – actual time taken to transfer data (depends on disk speed and amount of data)
- **Disk Access Time** – total time to access data
= Seek Time + Rotational Latency + Transfer Time
- **Disk Bandwidth** – total data transferred divided by total transfer time
- **Disk Response Time** – average waiting time before a request gets serviced
- **Variance Response Time** – measures how different each request's waiting time is from the average response time. Lower variance means more consistent service for all requests.
- **Cylinder:** All platters are stacked vertically and the disk arm moves all heads together simultaneously. So if your data is spread across the same track number on different platters, all of it can be accessed without moving the arm at all, just switch between heads.



Why Disk Scheduling?

OS is responsible for using hardware as efficiently as possible. For disk drives, you should have a fast access time and a high disk bandwidth and a small seek time.

- Only one I/O request handled at a time → others wait in a queue
- Far-apart requests = excessive disk arm movement
- Hard disks are slow → efficient scheduling is critical

I/O request: Contains input/output mode, disk address, memory address, and number of sectors to transfer.

OS request queue: OS maintains a queue of disk requests for each disk/device.

Idle vs busy disk:

- If disk is idle, the request is serviced immediately.
- If disk is busy, requests must wait in the queue.

Disk scheduling algorithms are useful only when multiple requests are waiting in a queue.

- Drive controllers: Have small buffers and can also maintain their own I/O request queues.
- Disk scheduling algorithms: Different algorithms are used to decide the order of servicing disk requests.

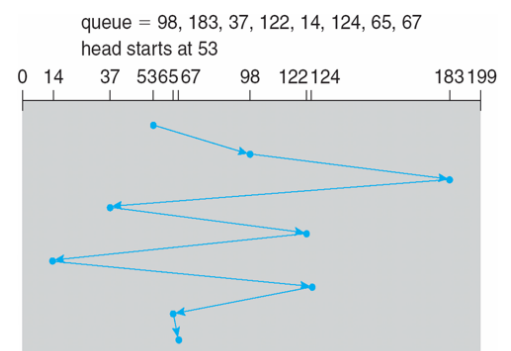
These scheduling concepts work for both single-platter (A hard disk containing only one magnetic disk platter for storing data) and multi-platter disks.

Scheduling Algorithms

Example queue: 98, 183, 37, 122, 14, 124, 65, 67 | Head starts at 53

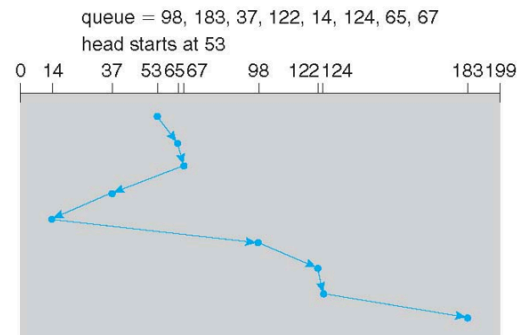
FCFS – First Come First Serve

- Requests serviced in arrival order
- Advantages: Fair, no starvation
- Disadvantages: Poor seek optimization, high response time
- Problem: causes large unnecessary head movement (e.g. 122 → 14 → 124)
- Implement FIFO queue of requests



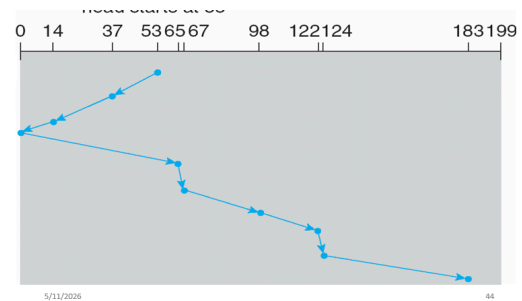
SSTF – Shortest Seek Time First

- Services the nearest request from current head position
- Sequence: 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183
- Total movement: 236 cylinders (vs FCFS: 640)
- Low average response time, high throughput
- Starvation – far requests may wait forever if nearby requests keep arriving



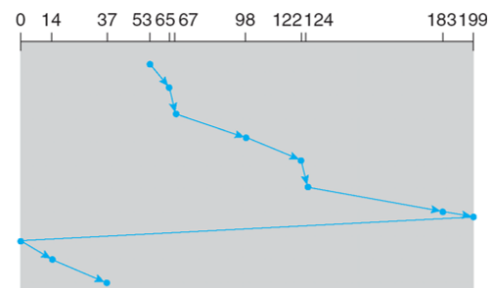
SCAN (Elevator Algorithm)

- Arm moves in one direction servicing requests, then reverses at end
- Sequence (moving toward 0): 37 → 14 → reverse → 65 → 67 → 98 → 122 → 124 → 183
- Total movement: 208 cylinders
- High throughput, low variance response time
- Requests just behind the head wait a long time



C-SCAN (Circular SCAN)

- Moves in one direction only
- At the end, jumps back to start without servicing on return
- More uniform waiting time than SCAN



LOOK

- Improved SCAN – reverses at the last request, not the disk end
- Example: Request sequence = {176, 79, 34, 60, 92, 11, 41, 114}

Sequence (head=50, moving right): 60 → 79 → 92 → 114 → 176 → 41 → 34 → 11

- Total seek: 291
- Better mean response time and variance than SCAN

C-LOOK (Circular LOOK)

- Circular version of LOOK – jumps back after last request
- Sequence: 60 → 79 → 92 → 114 → 176 → 11 → 34 → 41
- Total seek: 321
- Very popular in practice

Choosing an Algorithm

Situation	Best Choice
General use	SSTF or LOOK
Heavy load, avoid starvation	SCAN or C-SCAN

Performance also depends on request types and file allocation method.

Questions

Disk requests come to a disk driver for cylinders in the order 10, 22, 20, 2, 40, 6, and 38 at a time when the disk drive is reading from cylinder 20. The seek time is 6 ms/cylinder. The total seek time, if the disk arm scheduling algorithms is first-come-first-served is

- (A) 360 ms
- (B) 850 ms
- (C) 900 ms
- (D) None of the above

ans=d

GATE | GATE-CS-2014-(Set-1) | Question 65

Suppose a disk has 201 cylinders, numbered from 0 to 200. At some time the disk arm is at cylinder 100, and there is a queue of disk access requests for cylinders 30, 85, 90, 100, 105, 110, 135 and 145. If Shortest-Seek Time First (SSTF) is being used for scheduling the disk access, the request for cylinder 90 is serviced after servicing _____ number of requests.

- (A) 1
- (B) 2
- (C) 3
- (D) 4

Answer: (C)

Disk Management

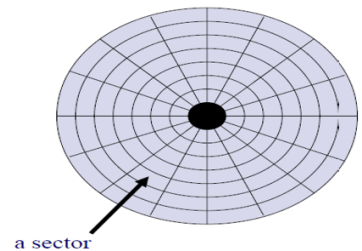
For I/O operations, data may be transferred as a stream of bytes or characters.

The OS is responsible for several disk management operations like

1. Disk initialization
2. Disk formatting
3. Booting
4. Badblock recovery

Disk initialization

- Divides disk into sectors readable/writable by controller
- Each sector = Header + Data + ECC (Error Correction Code)
- Common sector size: 512 bytes (also 256, 1024)
- Tests disk and maps logical blocks to defect-free sectors



Disk formatting

Partitioning

- Disk divided into logical partitions
- One partition may store OS, another user files

Logical formatting

- Creates file system structures (FAT/inodes) and empty directory

Booting

Disk management stores and loads the operating system from the bootable disk partition during startup.

Bad Blocks

- Defective sectors caused by disk wear or failure
- Controller maintains a bad-block list
- Sector sparing (forwarding) – bad sector is logically replaced by a spare sector, invisible to the OS

Module 4.2.2

I/O Management

The I/O Management is the OS component responsible for managing input/output devices and operations. I/O devices vary greatly: different devices need different control methods and performance management, and new devices keep getting added.

Types of I/O Devices

Storage/Machine-readable devices

- Include disks, tapes, sensors, controllers, and USB drives.
- Communication devices are used for remote communication, such as network cards and modems.
- Human-readable devices are used for user interaction, including monitors, keyboards, mice, and printers.
- Specialized devices are used in systems like aircraft or space shuttles: for example, a joystick that controls rudders and flaps through the computer.

The unit of transfer is either a byte/character stream or blocks depending on the device.

Different devices may also use different encoding schemes, parity methods, and error handling approaches.

Block

- Block-oriented devices store and transfer fixed-size blocks of data
- Transfers are made one block at a time
- Blocks can be accessed using a block number, such as disks and USB drives.

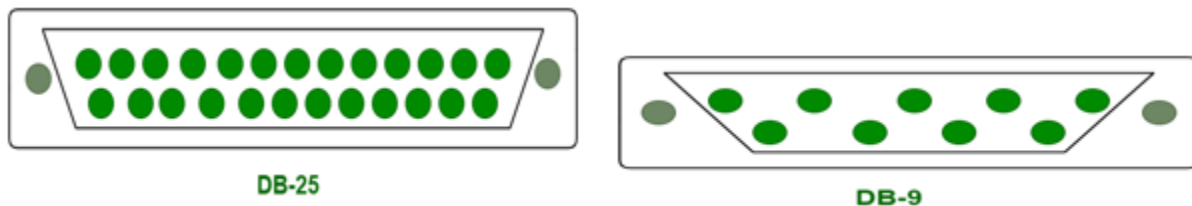
Stream

- Stream-oriented devices transfer a continuous stream of bytes without blocks
- Such as keyboards, mice, printers, and communication ports.

IO interface with Computer

Port

- A port is a connection point through which a device communicates with the computer.
- A serial port transfers data one bit at a time and uses a DB-9 connector.
- A parallel port transfers 8 bits simultaneously using multiple wires, making it faster than serial communication, and uses a DB-25 connector.



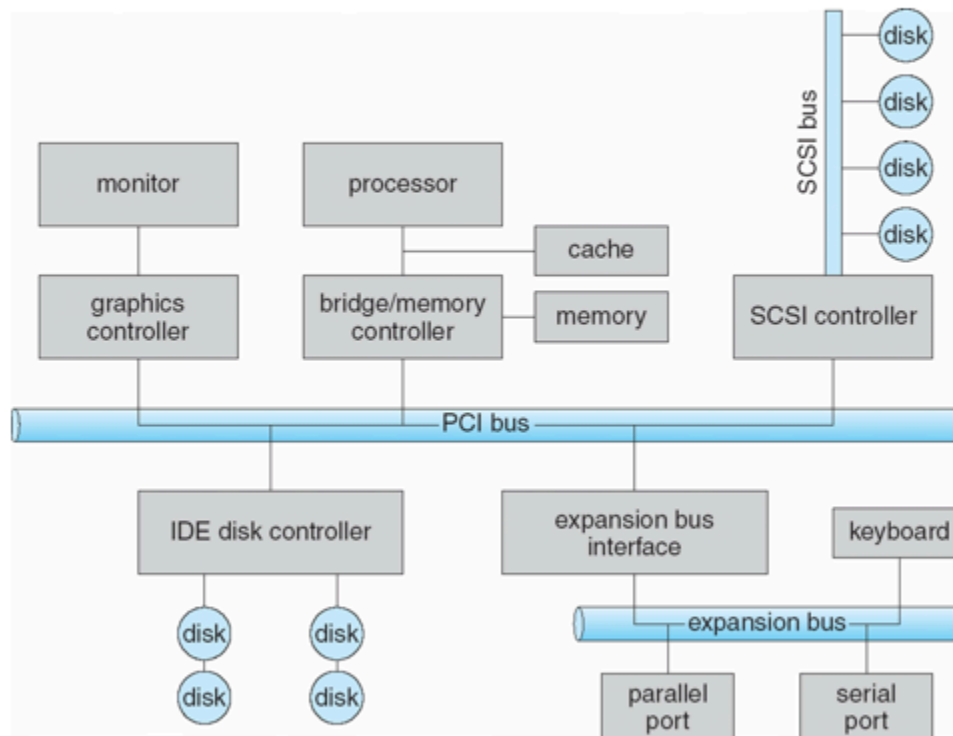
Difference between Serial port and Parallel Ports:

S.NO	SERIAL PORT	PARALLEL PORT
1.	Serial port is used to achieve serial transmission.	While parallel port is used to achieve parallel transmission.
2.	The transmission speed of serial port is comparatively low as compared to parallel port transmission speed.	While transmission speed of parallel port is higher than serial port transmission speed.
3.	In serial port communication less number of wires are used.	While in parallel port communication more number of wires are used as compared to serial port.
4.	A serial port is capable of delivering the single stream of data.	While a parallel port is capable of delivering multiple streams of data.
5.	Serial port send a bit after another bite at a time.	While parallel ports send multiple bits at once.
6.	In a serial port, male ports are involved.	While in a parallel port, female ports are involved.
7.	Serial ports are typically implemented in modems, connecting devices, security cameras and controllers.	Parallel ports are typically implemented in zip-drives, printers, hard drives, CD-ROM drives, etc.

Bus

- A bus is a shared set of wires used by devices for communication using fixed protocols and signals.
- In a daisy chain, devices are connected one after another (A -> B -> C -> computer).

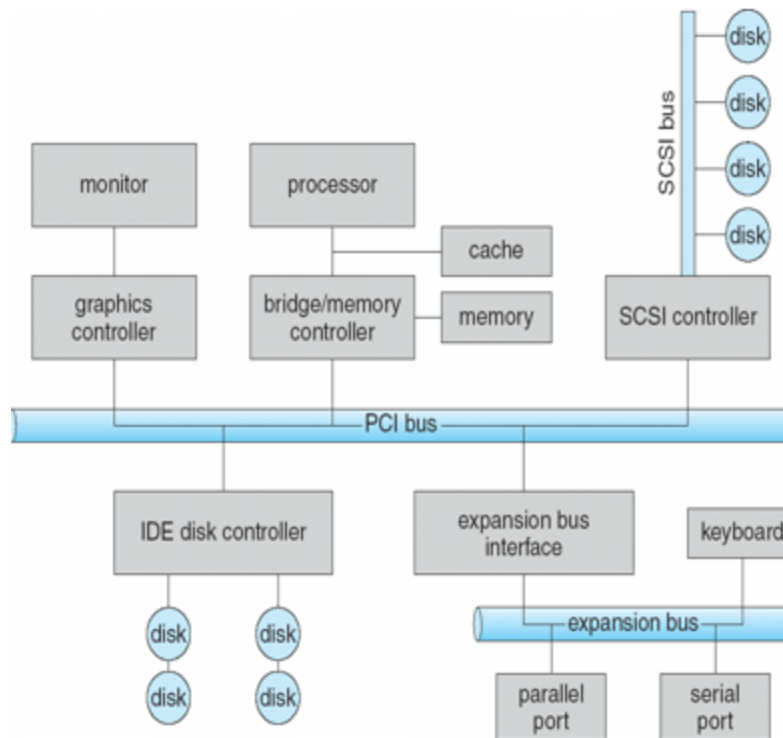
A Typical PC Bus Structure



Bus Types

1. PCI (Peripheral Component Interconnect) is a common high-speed PC system bus that connects the processor-memory subsystem to fast devices.
2. The expansion bus connects slower devices like keyboards and USB ports.

3. SCSI (Small Computer System Interface) is a parallel interface standard that allows faster communication with peripheral hardware.



Expansion Slots

1. ISA (Industry Standard Architecture) slots are old motherboard expansion slots used for display and sound cards, operating on a 16-bit bus. Identified by black colour.
2. PCI (Peripheral Component Interconnect) slots support sound cards, graphics accelerators, and DVD decoders on a 64-bit high-speed bus.
3. PCIe is the latest and fastest motherboard expansion interface and supports a full duplex serial bus. Full duplex means data can be sent and received at the same time, like a phone call where both people can talk and listen simultaneously. PCIe can send and receive data simultaneously.

Three I/O Techniques

1. Programmed I/O: the processor issues an I/O command and busy waits until the operation completes.

2. Interrupt-driven I/O: The processor issues an I/O command and continues working. If the I/O operation blocks the process, the OS pauses it and runs another process until the I/O completes.

Example

A program requests a file from disk. If non-blocking: CPU keeps running the same program while disk reads data. If blocking: OS pauses that program and runs another program until the disk operation finishes.

3. DMA (Direct Memory Access): a DMA module transfers data directly between memory and the I/O device with minimal CPU involvement. This is the dominant transfer method in modern operating systems.

DMA in Detail

The DMA controller is a hardware device that allows direct memory-device communication without continuous CPU involvement. During a DMA transfer, the DMA controller takes control of the system bus while the CPU stays mostly idle.

The processor sends the DMA controller a read/write request, the I/O device address, the memory address, and the number of words to transfer. The DMA controller then transfers the entire block directly between the device and memory, one word at a time. After the transfer is complete, the DMA controller sends an interrupt to the processor. The CPU is involved only at the start and end of the transfer.

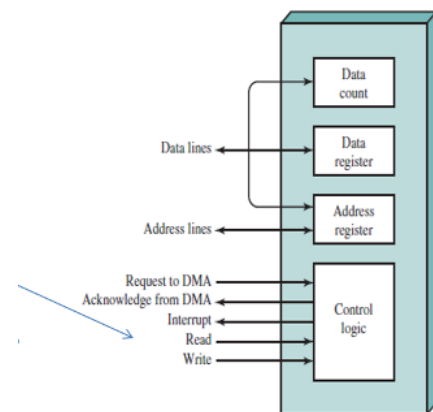


Figure 11.2 Typical DMA Block Diagram

GATE | GATE-CS-2016 (Set 1) | Question 41

The size of the data count register of a DMA controller is 16 bits. The processor needs to transfer a file of 29,154 kilobytes from disk to main memory. The memory is byte addressable. The minimum number of times the DMA controller needs to get the control of the system bus from the processor to transfer the file from the disk to main memory is _____

- (A) 3644
- (B) 3645
- (C) 456
- (D) 1823

Answer: (C)

Explanation: Size of data count register of the DMA controller = 16 bits

Data that can be transferred in one go = 2^{16} bytes = 64 kilobytes

File size to be transferred = 29154 kilobytes

So, number of times the DMA controller needs to get the control of the system bus from the processor to transfer the file from the disk to main memory = $\text{ceil}(29154/64) = 456$

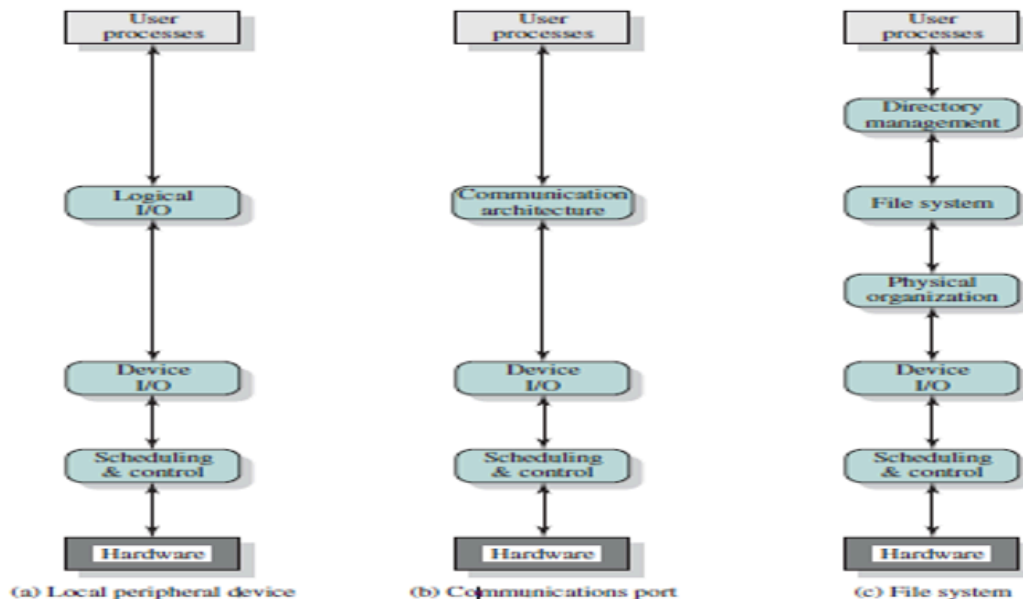
I/O Design Objectives

- **Efficiency:** I/O devices are much slower than the CPU and memory, which creates a bottleneck. The solution is multiprogramming: while one process waits for I/O, another process executes.
- **Generality:** the OS should handle all devices uniformly for simplicity and reliability. The challenge is that devices have very different characteristics. The solution is a hierarchical modular design that hides low-level device details. Upper OS layers and user processes see only simple functions like read, write, open, close, lock, and unlock.

Hierarchical I/O Structure

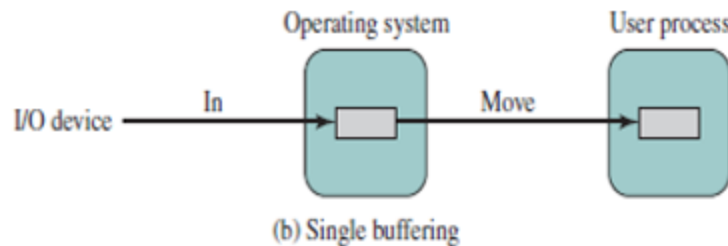
The OS is divided into layers where each layer uses the services of the layer below it and provides services to the layer above. The goal is that changes in one layer should not affect the others.

- Logical I/O treats the device as a logical resource and handles general operations like open, read, and write.
- Device I/O converts those logical operations into actual device instructions and controller commands.
- Scheduling and control handles queueing, scheduling, interrupts, and I/O status reporting.
- Buffering may be used at various levels to improve utilization and performance.

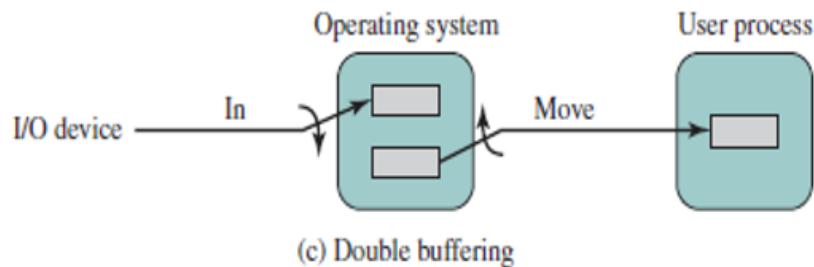


Buffering

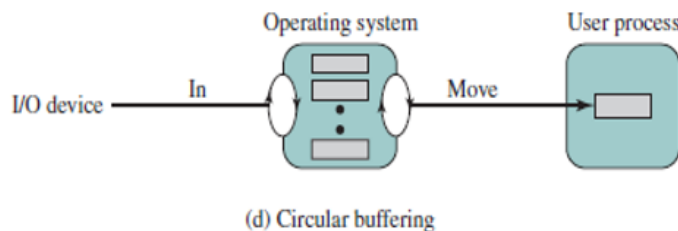
1. Single Buffer: the OS allocates one system buffer for an I/O operation. Reading ahead (anticipated input) means the next block is requested immediately, expecting it will be needed soon. Single buffering is faster than no buffering.



2. Double Buffer: two buffers are used. One buffer is used by the process while the OS fills or empties the other. This is called buffer swapping.



3. Circular Buffer: more than two buffers are used to handle heavy burst I/O. It is based on the bounded-buffer producer-consumer model.



Buffering smooths out peaks in I/O demand. However, if process demand consistently stays higher than device speed, the buffers eventually fill up

and the process must wait. In a multiprogramming environment, buffering improves both OS efficiency and overall process performance.

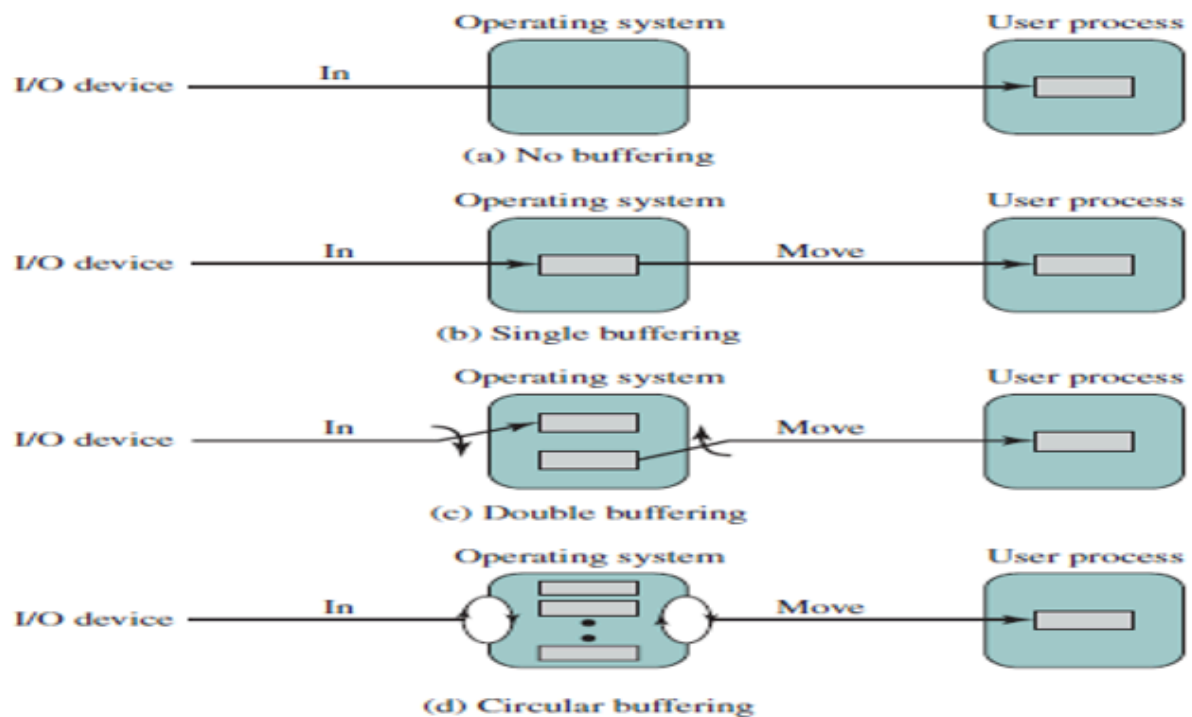


Figure 11.5 I/O Buffering Schemes (Input)

Module 5

Memory Management

1. What is memory?

Memory (also called main memory, RAM, primary memory, or physical memory) is a large array of bytes, each with its own address. It stores both instructions and data. The CPU can directly access only two places: main memory and its own registers. Disk addresses cannot be used directly by instructions.

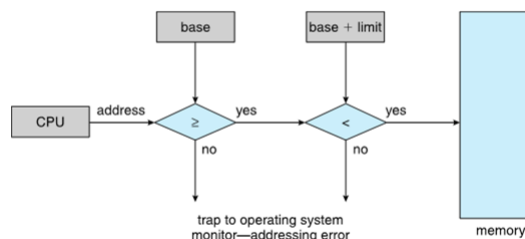
Instruction execution cycle

Instruction Fetch → Instruction Decode → Operand fetch → Execute

The program counter (PC) stores the address of the next instruction to execute. The memory unit only sees an address plus a read/write request – it has no awareness of what the data means.

Memory problems and solutions

- Speed problem – Main memory access takes many CPU cycles, causing CPU stall. Solved by cache memory – fast memory placed between CPU and RAM.
- Protection problem – OS and user processes must be isolated from each other. Base register and limit register ensure the process can access only addresses within the legal range.
 - Base Register: Holds the smallest legal physical memory address a process can access.
 - Limit Register: Specifies the size/range of legal memory addresses accessible to the process.
 - Example: If the base register holds 300040 and the limit register is 120900, then the program can legally access all addresses from 30 0040 through 420939 (inclusive).



- Size problem – Not all programs fit in memory at once. Solved by swapping (Temporarily moving a process from main memory to disk and bringing it back later when needed) and virtual memory.

2. Hardware access protection

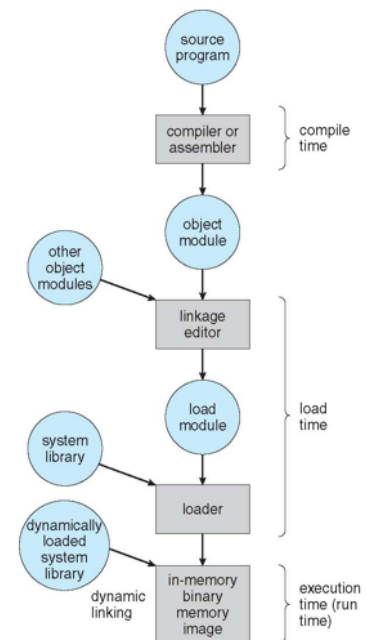
- The base and limit registers can only be loaded by the OS
- This is because the loading can only be done by privileged instructions. And privileged instructions are only operated in the kernel mode. Only OS operates in kernel mode.
- Allows the operating system to change the value of registers. However the user programs cannot change the contents of the registers.
- The OS can load the user programs to user memory and can dump those out in case of errors.

3. Address binding

Programs on disk are stored as executable files. Before they run, their symbolic addresses must be mapped to actual physical memory addresses. This mapping is called **address binding**.

1. Symbolic address → e.g. count (name in source code)
2. Relocatable address → e.g. 14 bytes from start (relative)
3. Absolute address → e.g. 74014 (actual physical address)

Addresses in the source program are symbolic. Compiler turns this into relocatable. The linker or loaders turn relocatable addresses to absolute addresses.



Binding stages

Address binding of instructions and data to memory addresses can happen at three different stages.

Stage	When	Notes

Compile-time	At compilation	Compiler generates absolute code. Absolute code is code where memory addresses are already fixed beforehand and cannot easily change later. If location changes, must recompile.
Load-time	When loaded into memory	Compiler generates relocatable code. If location changes, must reload.
Execution-time	At runtime	Process can move during execution. Needs hardware (MMU) support of base and limit registers

Logical vs. physical addresses

The CPU generates a logical address (virtual address). The actual location in RAM is the physical address. The physical address is what the memory unit sees. They are equal in compile-time and load-time binding. They differ in execution-time binding.

MMU

The MMU (Memory Management Unit) is hardware that translates logical addresses to physical addresses at runtime using a relocation register. Base register is called the relocation register.

Physical address = logical address + relocation register value

Example: Relocation register = 14000. Logical 0 → Physical 14000. Logical 346 → Physical 14346. The user program never sees actual physical addresses.

4. Dynamic loading & linking

Routine: A small section/function of a program designed to perform a specific task.

- **Dynamic loading** – A routine is loaded into memory only when it is actually called. Unused routines (like rare error handlers) are never loaded, saving memory. Implemented through program design – no special

OS support required. It is the responsibility of the users to design their programs to take advantage of such a method.

When a routine calls another routine, the caller checks if the callee is already loaded. If no, then the dynamic loader is called to load and update the program's address table. Then the control is passed to the newly loaded routine.

- **Static linking** – Libraries are combined into the executable before execution. Every program carries its own copy of the library – wastes disk and memory.
- **Dynamic linking** – Library linking is delayed until runtime.

The stub is a small piece of code that indicates how to locate the appropriate memory-resident library routine or how to load the library if the routine is not already present.

A stub checks if the routine is in memory; if not, it loads it and replaces itself with the routine's address. Library updates are automatically used by all programs without relinking.

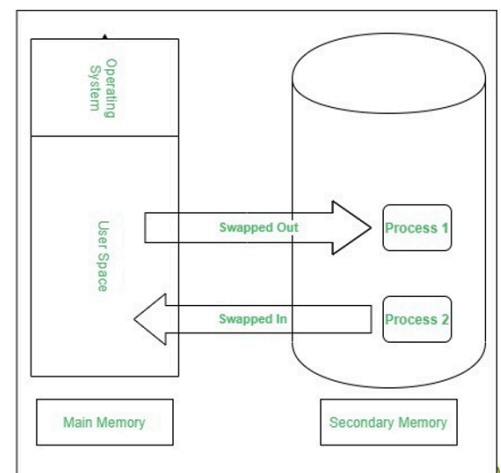
5. Swapping

When memory is full, the OS temporarily moves a process out to a fast disk area called the backing store, freeing RAM for another process. The process is later swapped back in to continue execution.

The backing store is a fast disk large enough to store copies of all memory images. It must provide direct access to these memory images. A memory image is the complete contents/state of a process in memory at a particular moment.

- Swapper manages memory– decides which process to swap in/out.
- Dispatcher manages CPU– selects which process gets the CPU next.

The dispatcher checks to see whether the next process in the queue is in memory. If it is not, and if there is no free memory region, the swapper swaps out a process currently in memory and swaps in the desired process. The context switch time in such a swapping system is fairly high.



For efficient CPU utilization, the execution time for each process should be substantially larger than the time required to swap the process in and out.

- Roll out / roll in – swapping a low-priority process out to run a higher-priority one. After the high priority process is done, the low priority process is swapped back in.
- Swap time depends mainly on the amount of memory being transferred. System maintains a ready queue of ready-to-run processes which have memory images on disk.

Normally, a process that was earlier swapped out will go back to its same memory space it occupied previously. This is decided by the method of address binding.

- With execution-time binding, a swapped process can be loaded back into a different memory location.
- With compile/load-time binding, it usually needs to return to the same location.

6. Contiguous allocation

Each process is stored in one continuous block of memory. Executing process must entirely be loaded into the main memory. OS needs to allocate main memory in the most efficient way since it needs to be given to both OS and processes. The main memory is divided into two partitions: one for the resident OS and one for the user processes. The OS is either in the low memory or in the high memory.

The major factor affecting this decision is the location of the interrupt vector. Since the interrupt vector is often in low memory, programmers usually place the operating system in low memory as well. Interrupt vector: A list telling the CPU which function/program to run when a specific interrupt happens. Eg:

Keyboard pressed → CPU checks interrupt vector → jumps to keyboard-handling code.

Memory mapping and protection

Memory protection is done using a relocation register and a limit register so that user processes cannot access or modify OS memory or other processes' memory.

- The relocation register stores the smallest physical address allocated to the process. It acts as the starting address of that process in memory.
- The limit register stores the size/range of logical addresses the process is allowed to access. Every logical address generated by the process must be smaller than the limit value, otherwise the OS throws an error/trap.
- Each logical address must be less than the limit register. The MMU maps the logical address dynamically by adding the value in the relocation register.

Fixed partitioning

Memory is divided into a fixed number of partitions before any process runs. Size of each partition may or may not be the same. Each partition holds one process, so the number of concurrent processes is limited. Each partition is dedicated to one process UNTIL the process terminates/releases the partition.

In this Multiple partition method, when a partition is free, a process is selected from the input queue and is loaded into the free partition.

When a process arrives and needs memory, the system searches the set for a hole that is large enough for this process. If the hole is too large, it is split into two parts. One part is allocated to the arriving process; the other is returned to the set of holes. If the hole is too small, the OS waits for a large enough hole to be available or skips down the input queue to find a smaller memory requirement.

When a process terminates, it releases its block of memory, which is then placed back in the set of holes. If the new hole is adjacent to other holes, these adjacent holes are merged to form one larger hole. The system checks for any new processes that might benefit from this hole.

Disadvantages

- Causes internal fragmentation
- wasted space inside a partition when the process is smaller than the partition.

Variable/Dynamic Partitioning

Partitions are created dynamically to match each process's size, reducing internal fragmentation. Free blocks are called holes. The OS maintains a list of allocated partitions and free holes.

How to satisfy a request of size n from a list of free holes?

Strategy	How it works	Result
First-fit	Allocate the first hole large enough. Searching can start either at the beginning of the set of holes or at the location where the previous first-fit search ended.	Fastest generally
Best-fit	Allocate the smallest suitable hole. Search the entire list linearly except for when it's ordered by size.	Smallest leftover fragment
Worst-fit	Allocate the largest hole. Searching can start either at the beginning of the set of holes or at the location where the previous first-fit search ended.	Largest leftover fragment – generally worst performance

Neither first fit nor best fit is clearly better than the other in terms of storage utilization, but first fit is generally faster.

7. Fragmentation

As processes are loaded and removed from memory, the free memory space is broken into little pieces.

1. External fragmentation – Enough total free memory exists, but it is scattered in small non-contiguous holes– no single hole is large enough for the process.
2. Internal fragmentation – Allocated partition is larger than the process needs. The leftover space inside the partition is wasted.

The first fit analysis reveals that given N allocated blocks, roughly $0.5N$ blocks are lost to external fragmentation. This is called the 50% rule. So about one-third of memory is wasted.

Compaction

All processes are shifted together in memory to merge scattered free holes into one large block. Only possible with execution-time binding, since processes can be relocated at runtime. Can be expensive.

Non-contiguous memory allocation

If we allow the logical address space to be non contiguous, we can allow the physical memory to be allocated as needed. But the trade-off is extra overhead: the system now needs address translation structures (like page tables/segment tables) to keep track of where each piece is stored, which uses extra memory and processing time. We can do this by paging and segmentation.

Fork

fork()

1. What is fork()?

fork() is a system call used in Unix/Linux operating systems to create a new process. The process that calls fork() is called the parent process. The newly created process is called the child process. The child is an almost exact duplicate of the parent.

2. What gets copied?

When fork() is called, the child process gets a copy of:

- Code (program instructions)
- Data (variables)
- Stack
- Heap
- File descriptors (open files, sockets)
- Program counter – so the child starts executing from the same point as the parent

The child does NOT share memory with the parent – it gets its own separate copy.

3. Return value of fork()

fork() returns a different value depending on who is reading it:

Returned to	Value	Meaning
Parent process	Child's PID (positive integer)	So parent knows the child's ID

Child process	0	So child knows it is the child
On failure	-1	Process could not be created

This is how the same code can make the parent and child behave differently – they check the return value and branch accordingly.

4. Basic example

```
pid_t pid = fork();

if (pid < 0) {
    // fork failed
} else if (pid == 0) {
    // child process runs this
} else {
    // parent process runs this
    // pid = child's PID
}
```

After `fork()`, both parent and child run concurrently. The order in which they execute depends on the CPU scheduler – there is no guarantee which runs first.

5. Process IDs (PID)

Every process has a unique process ID (PID).

- `getpid()` – returns the calling process's own PID.
- `getppid()` – returns the parent's PID.

The child's PID is always a new unique number. The child's PPID (parent PID) is the parent's PID.

6. Copy-on-write (COW)

Copying the entire parent memory space every time `fork()` is called is expensive. Modern OS uses copy-on-write – parent and child initially share

the same memory pages. A copy is only made when either process tries to write to a page.

This makes `fork()` much faster since most child processes don't modify all of the parent's memory.

7. What happens after `fork()`?

After forking, the child usually does one of two things:

Option 1 – `exec()`: Replace itself with a completely different program using `exec()`. The child's memory is wiped and replaced with the new program. This is how shells launch commands – fork a child, then `exec` the command.

Option 2 – continue as a copy: The child continues running the same code as the parent but as a separate process (e.g. in web servers, each child handles a separate client request).

8. Zombie and orphan processes

Zombie process – A child that has finished executing but its exit status has not been collected by the parent yet. It stays in the process table as a zombie until the parent calls `wait()`.

Orphan process – A child whose parent has terminated before it. The orphan is adopted by the `init` process (PID 1), which collects its exit status when it finishes.

9. `wait()` and `waitpid()`

After forking, the parent can call `wait()` to pause and wait for the child to finish. This prevents zombie processes.

```
pid_t pid = fork();

if (pid == 0) {
    // child does its work
    exit(0);
} else {
    wait(NULL); // parent waits for child to finish
}
```

`waitpid(pid, status, options)` is a more specific version – the parent can wait for a specific child by PID.

10. `fork()` vs. `exec()` vs. `clone()`

Call	What it does
<code>fork()</code>	Creates a copy of the current process
<code>exec()</code>	Replaces current process with a new program
<code>clone()</code>	Like <code>fork()</code> but with fine control over what is shared (used for threads)

A common pattern is `fork()` followed immediately by `exec()` – create a child, then replace it with the desired program.

11. Key points to remember

- `fork()` creates a child that is a copy of the parent.
- Both run concurrently after the fork – scheduler decides the order.
- Return value tells each process who they are.
- Copy-on-write makes forking efficient.
- Always use `wait()` to avoid zombie processes.
- `fork()` + `exec()` is the standard way to launch new programs in Unix.

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main() {
```

```
    pid_t pid = fork(); // create child process
```

```
    if (pid < 0) {
```

```
        // fork failed
```

```
        printf("Fork failed\n");
```

```
        return 1;
```

```
    } else if (pid == 0) {
```

```
// child process

printf("Child: PID = %d\n", getpid());

exec("/bin/ls", "ls", NULL); // replace child with ls program

// anything after exec() never runs


} else {

    // parent process

    printf("Parent: PID = %d, Child PID = %d\n", getpid(), pid);

    wait(NULL); // wait for child to finish

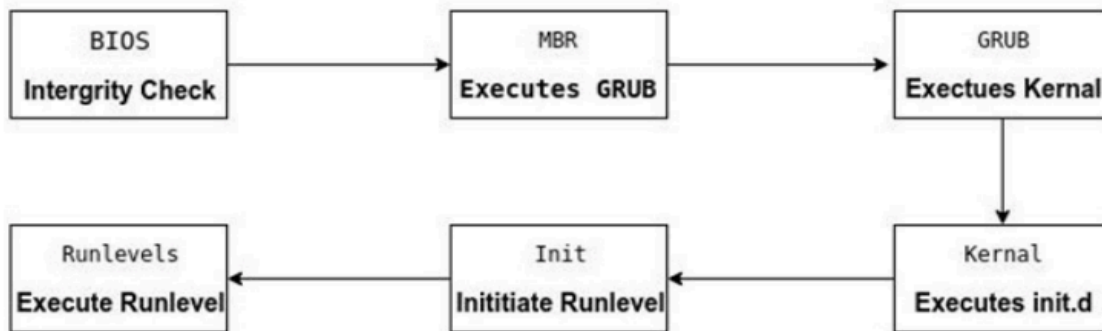
    printf("Parent: child done\n");

}

return 0;

}
```

Skip



Master Boot Record (MBR)

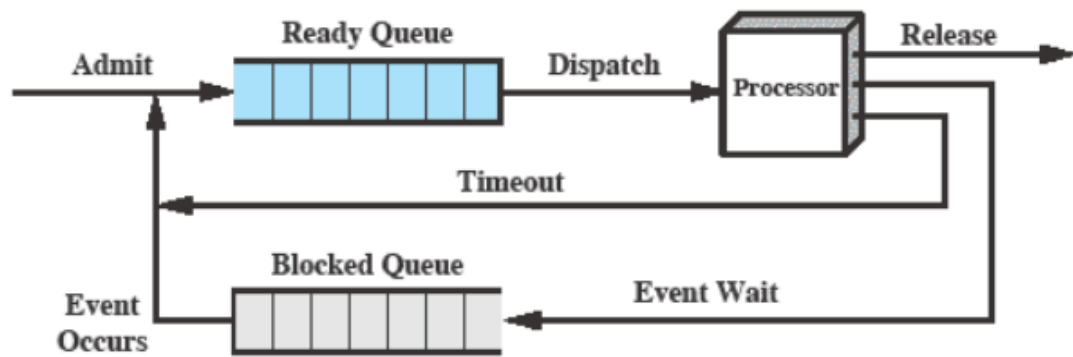
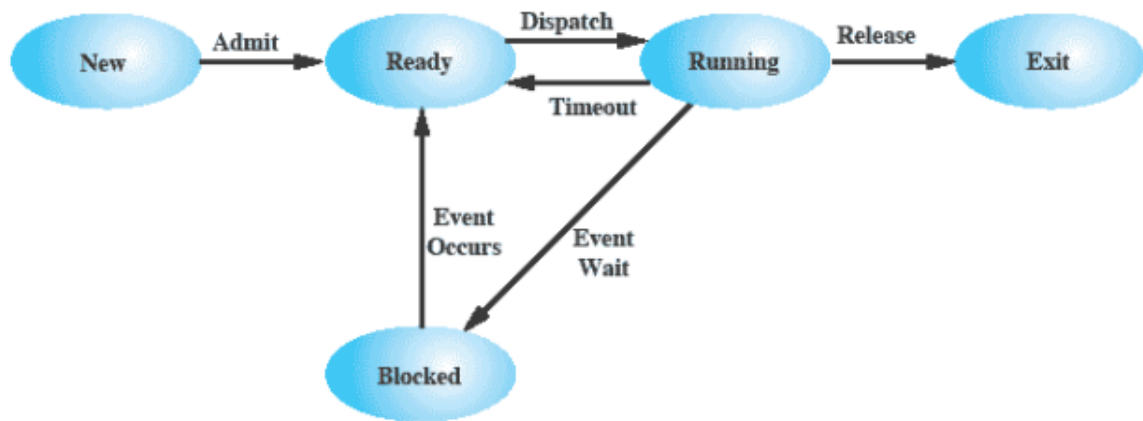
It is the information which is in the **first sector of any hard disk**

It holds information about GRUB (or LILO in old systems).

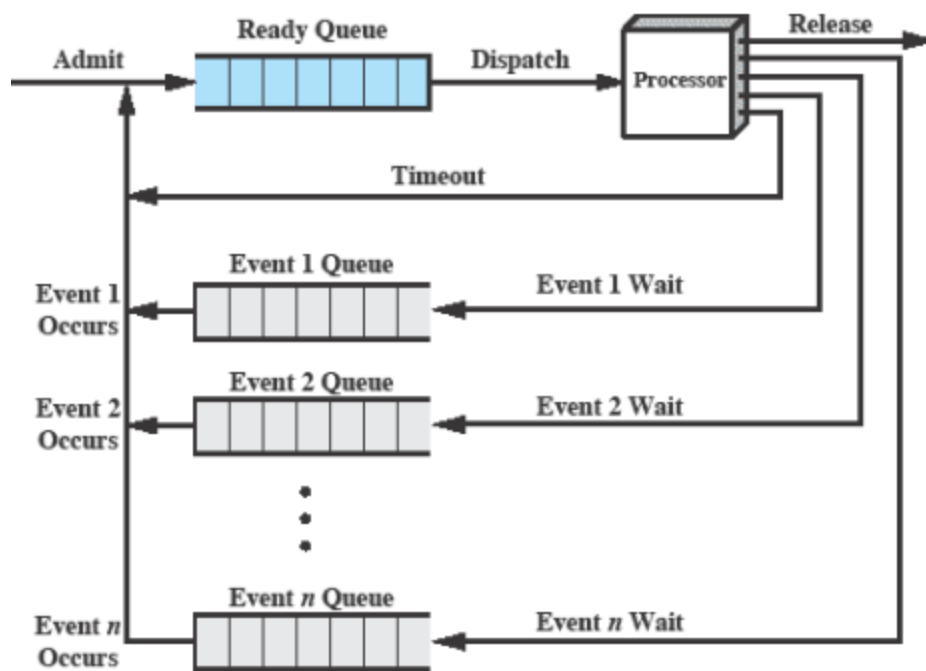
Current OS designs

- **Windows NT and successors (2000, XP, Vista, 7, 8, 10, 11):** Hybrid kernel with microkernel elements.
- **macOS (and iOS, iPadOS, watchOS, tvOS):** Based on the XNU kernel, combining Mach microkernel and FreeBSD components.
- **Linux distributions:** Monolithic kernel with dynamically loadable modules, offering hybrid-like flexibility.
- **Solaris:** Primarily monolithic but incorporates some microkernel principles with loadable kernel modules.
- **Android:** Built on the Linux kernel, following a similar modular approach.

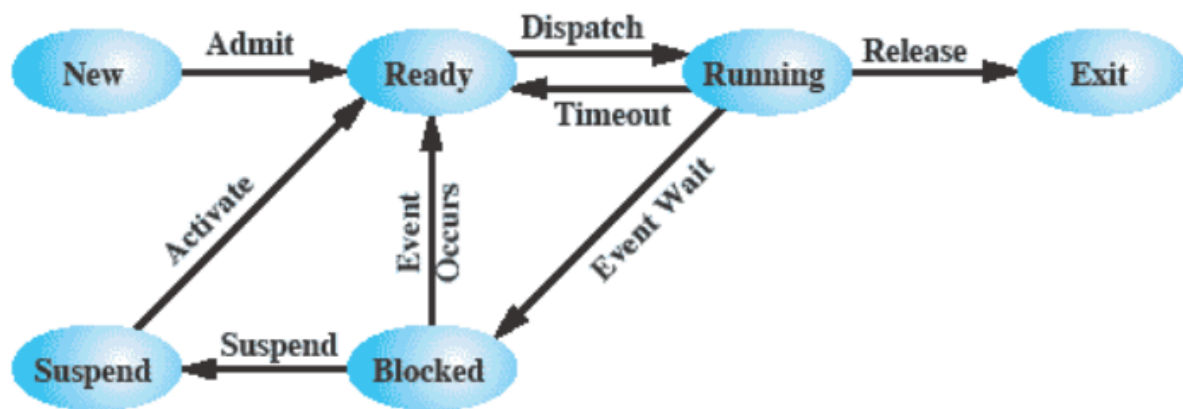
ALL DIAGRAMS



(a) Single blocked queue



(b) Multiple blocked queues

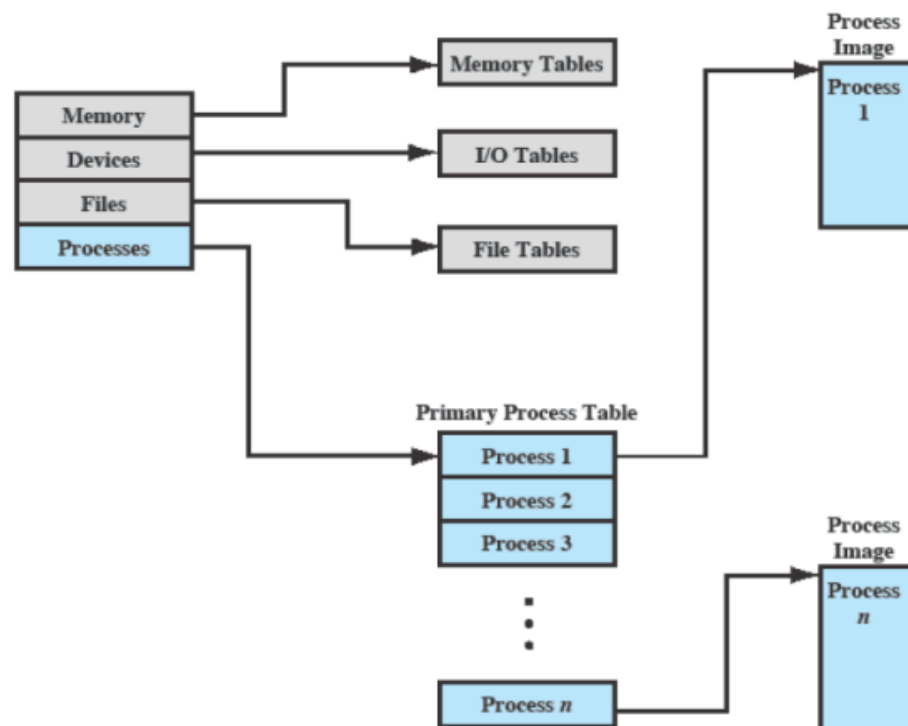


(a) With One Suspend State



(b) With Two Suspend States

OS Control Tables: General structure



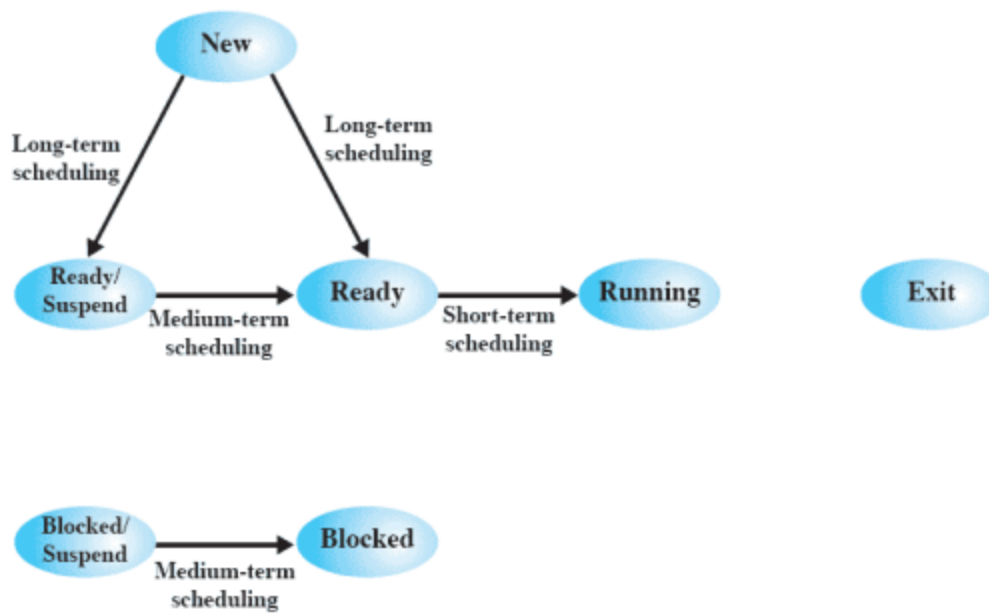


Figure 9.1 Scheduling and Process State Transitions

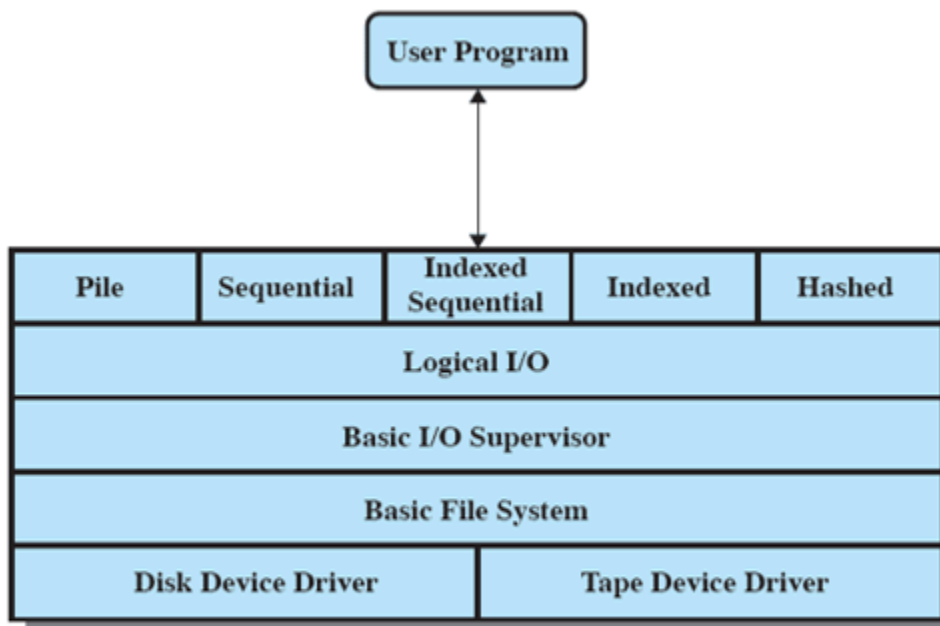


Figure 12.1 File System Software Architecture

Address binding

